

Answers to

Theory of Computation

Jim Hefferon
<https://hefferon.net/computation>



About the answers

A person who provides answers to text exercises has decisions to make: some exercises or all? or maybe all exercises to instructors only? complete worked answers or summaries? if summaries, what about exercises involving proofs?

I find that when I write a book, if I generate the complete answers then that makes the questions better. In working through them I am checking that the section contains all the needed prerequisites and also that the question is clear. I often go back to adjust either the question or the section body. Besides, I enjoy the process and there may as well be one person having a good time. So I have the answers.

However, whether to give them out is something to consider. The point of exercises is for students to do them. Providing the answers risks that some students will just read the questions and read their answers and proceed feeling that they understand, which they do not. Weighed against that is the worry that providing not-fully-worked answers, or hints, or only some answers, can leave an honestly-trying person who has occasional trouble with no way to get what it is that they didn't see.

I am sometimes asked to provide for download the answers to only some exercises, and to reserve the rest for instructors. Putting aside the increased email burden on me for something that I give away, this is impractical because I have no way of verifying that someone who contacts me is who they claim to be. But there is also a larger point. This request assumes that readers are using the text in a college or university. I find that many are outside of that system, and just want to learn. Finally, I'll also remark that the people asking this have a different idea than I do of the Internet. My experience has been that, as with a traditionally published Calculus text, any student who looks online for the complete answer document will be able to find it within minutes. So the belief that somehow the other texts that students have do not also give them access to all exercises seems to me to be mistaken.

In short, I provide complete answers to all exercises, including proofs.

Note: This PDF matches the book version compiled on 2025-Jul-26. If you put that book version and this document in the same directory then clicking on an exercise number in the book will take you to its answer here, and clicking on the answer number will take you back to the book.

Jim Hefferon
University of Vermont

Cover: Flauros, the sixty-fourth demon and the Duke of Hell, who gives true answers of all things past, present, and future. Although, you should verify what he says for yourself. (Unless you can get him inside a triangle and then you're good. It is unclear whether that is a permanent triangle, or just one chalked out on the floor, or a triangle of roads, or rivers, or planets.) From *Dictionnaire Infernal*, Jacques Albin Simon Collin de Plancy, 1863.

Chapter V: Computational Complexity

- V **V.1.31** True. Informally, Big $\mathcal{O}$ means something like “grows at a rate less than or equivalent to.” Growing at a rate less than or equivalent to the rate of x^2 implies growing at a rate less than or equivalent to the rate of x^3 .

More formally, assume that f is $\mathcal{O}(x^2)$. Then there are constants $N, C \in \mathbb{R}$ so that if $x \geq N$ then $f(x) \leq C \cdot x^2$. Since $x^3 \geq x^2$ for all $x \geq 1$, we have: if $\hat{x} \geq \max(N, 1)$ then $f(\hat{x}) \leq C \cdot \hat{x}^3$. Thus f is $\mathcal{O}(x^3)$.

- V **V.1.32** First, the assertion that “The algorithm has running time $\mathcal{O}(n^2)$ ” does not mean that the runtime function is quadratic, since if $f(n) = n$ then f is $\mathcal{O}(n^2)$.

Second, even if the running time is quadratic, it may involve constants and lower order terms. Thus, $g_0(n) = 1000 \cdot n^2$ is $\mathcal{O}(n^2)$, as is $g_1(n) = n^2 + 3n + 42$. But neither $g_0(5)$ nor $g_1(5)$ equals 25.

(Still another issue is that even if it is quadratic then the runtime behavior is quadratic only for sufficiently large inputs. So unless we know that 5 is greater than the N of the definition, there is no reason at all to think about 25.)

- V **V.1.33** Perhaps they mean that for their problem the algorithm that they have is $\mathcal{O}(n^3)$. (If they mean to associate the runtime with the problem not the algorithm then perhaps they mean that the best algorithm known is $\mathcal{O}(n^3)$?)

- V **V.1.34**

- (A) Because 5 in binary is 101, it takes three bits.
 (B) The number 50 in binary is 11 0010 so it takes six bits.
 (C) In binary the number 500 is 1 1111 0100 so it takes nine bits.
 (D) The binary equivalent of 5 000 is 1 0011 1000 1000, which is thirteen bits.

This table verifies these using the formula $\text{bits}(n) = 1 + \lfloor \lg(n) \rfloor$.

Integer n in decimal	5	50	500	5 000
Binary representation	101	11 0010	1 1111 0100	1 0011 1000 1000
$\lg(n)$	2.322	5.644	8.966	12.288
$1 + \lfloor \lg(n) \rfloor$	3	6	9	13

- V **V.1.35** The second implication is true while the first is not.

Recall the intuition that ‘ f is $\mathcal{O}(g)$ ’ means something like “ f ’s growth rate is less than or equivalent to g ’s,” while ‘ f is $\Theta(g)$ ’ means something like “ f ’s growth rate is equivalent to g ’s.” So the second statement makes sense because if f and g grow at equivalent rates then that implies that f ’s rate is less than or equivalent to g ’s. Arguing precisely, the second is true because by definition if f is $\Theta(g)$ then both f is $\mathcal{O}(g)$ and g is $\mathcal{O}(f)$.

As to the first statement, the intuition is that assuming that f ’s growth is less than or equivalent to g ’s does not force that the two are equivalent. That is, there are functions f, g so that f is $\mathcal{O}(g)$ but f is not $\Theta(g)$. One such pair is $f(n) = n$ and $g(n) = n^2$.

- V **V.1.36** The correct answer is ‘None’. We can’t apply Big $\mathcal{O}$ to a particular input size. Big $\mathcal{O}$ is about how the function grows as the input size grows and does not apply to individual inputs. For one thing, if 61 669 is not greater than the definition’s N then the statement of quadratic-ness doesn’t apply at all.

Comment. Sometimes in practice people refer to runtime for a $\mathcal{O}(n^2)$ algorithm, or the code implementing it, by saying something like, “the runtime when $n = 60\,000$ should be in the neighborhood of quadruple the time for $n = 30\,000$.” They are either being a bit sloppy or they mean that they are familiar with the algorithm or code and that the runtime behavior is already roughly quadratic by the input size n . But with only the information given in this question we cannot say that.

- V **V.1.37** We use the principles for simplifying Big $\mathcal{O}$ expressions from page 268.

- (A) $\mathcal{O}(n^2)$

- (B) $\mathcal{O}(2^n)$
- (C) $\mathcal{O}(n^4)$
- (D) $\mathcal{O}(\lg n)$

V V.1.38

- (A) For sufficiently large input arguments n this function is $f(n) = 0$. So it is $\mathcal{O}(1)$.
- (B) For sufficiently large inputs n this function is $f(n) = n^2$. So it is $\mathcal{O}(n^2)$.
- (C) For sufficiently large n this function is $f(n) = \lg(n)$. So it is $\mathcal{O}(\lg(n))$.

V V.1.39 Because we will apply L'Hôpital's Rule, we convert from n 's to x 's, signaling we must take them to be real number functions.

- (A) To apply the theorem consider the limit of the ratio.

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{\ln(x) + 6}$$

This is an " ∞/∞ " limit. Apply L'Hôpital's Rule.

$$\lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{\ln(x) + 6} = \lim_{x \rightarrow \infty} \frac{9x^2 + 2}{1/(\ln(2)x)} = \ln(2) \cdot \lim_{x \rightarrow \infty} \frac{x \cdot (9x^2 + 2)}{1} = \infty$$

So g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$. In short, f 's growth rate is strictly bigger than g 's.

- (B) This is of type " ∞/∞ ." Use L'Hôpital's Rule multiple times.

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{x + 5x^3} = \lim_{x \rightarrow \infty} \frac{9x^2 + 2}{1 + 15x^2} = \lim_{x \rightarrow \infty} \frac{18x}{30x} = \lim_{x \rightarrow \infty} \frac{18}{30}$$

Conclude that f is $\Theta(g)$ (we can also state that as g is $\Theta(f)$). Briefly, f 's growth rate is equivalent to g 's.

- (C) This is the limit of the ratios.

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{(1/2)x^3 + 12x^2}{x^2 \ln(x)}$$

We could start by simplifying the fraction, canceling x^2 from the numerator and the denominator. However, we choose to instead start by applying L'Hôpital's Rule to this " ∞/∞ " limit. Note the use of the Product Rule in the denominator.

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{(1/2)x^3 + 12x^2}{x^2 \ln(x)} &= \lim_{x \rightarrow \infty} \frac{(3/2)x^2 + 24x}{2x \ln(x) + x} = \lim_{x \rightarrow \infty} \frac{(3/2)x^2 + 24x}{x(2 \ln(x) + 1)} = \lim_{x \rightarrow \infty} \frac{(3/2)x + 24}{2 \ln(x) + 1} \\ &= \lim_{x \rightarrow \infty} \frac{3/2}{2/x} = \lim_{x \rightarrow \infty} \frac{3x}{4} = \infty \end{aligned}$$

So g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.

- (D) The limit is " ∞/∞ ." L'Hôpital's Rule gives this.

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{\lg(x)}{\ln(x)} = \lim_{x \rightarrow \infty} \frac{(1/x) \cdot (1/\ln(2))}{(1/x)} = \lim_{x \rightarrow \infty} \frac{1}{\ln(2)} \cdot \frac{(1/x)}{(1/x)} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} 1$$

So they grow at equivalent rates: f is $\Theta(g)$.

- (E) Recognize this as an " ∞/∞ " limit.

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} &= \lim_{x \rightarrow \infty} \frac{x^2 + \lg(x)}{x^4 - x^3} = \lim_{x \rightarrow \infty} \frac{2x + 1/(\ln(2) \cdot x)}{4x^3 - 3x^2} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{2 \ln(2) \cdot x^2 + 1}{4x^4 - 3x^3} \\ &= \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{4 \ln(2) \cdot x}{16x^3 - 9x^2} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{4 \ln(2)}{16x^2 - 9x} = 0 \end{aligned}$$

So f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

(F) We consider the ratio of the functions.

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{55}{x^2 + x}$$

This is not an “ ∞/∞ ” limit (it is instead “ $55/\infty$ ”) so L'Hôpital's Rule doesn't apply. But that doesn't matter because the limit is easy. The limit is zero. Thus f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

- V **V.1.40** Note that $\cos(t)$ is bounded by 1. Also, $\lg(5^n) = n \cdot \lg(5)$. Thus, the ones with a growth rate less than or equivalent to n^2 's are the first, second, fourth, and fifth. The third one has a growth rate that is $\Theta(n^3)$, bigger than n^2 's.
- V **V.1.41** We can do these is with Lemma 1.25 and Table 1.27, and also with Theorem 1.15. Here we did the first two both ways, just to illustrate.

(A) True. By Lemma 1.25 this is the sum of a function that is $\mathcal{O}(n^2)$ with one that is $\mathcal{O}(n)$. By the same lemma and Table 1.27 the sum is $\mathcal{O}(n^2)$, and therefore is $\mathcal{O}(n^3)$.

Here is an alternate argument using Theorem 1.15.

$$\lim_{x \rightarrow \infty} \frac{5x^2 + 2x}{x^3} = \lim_{x \rightarrow \infty} \frac{10x + 2}{3x^2} = \lim_{x \rightarrow \infty} \frac{10}{6x} = 0$$

(B) False by Theorem 1.15.

$$\lim_{x \rightarrow \infty} \frac{2 + 4x^3}{\lg x} = \lim_{x \rightarrow \infty} \frac{12x^2}{(1/x)} = \lim_{x \rightarrow \infty} 12x^3 = \infty$$

Alternatively, we could apply Lemma 1.25 to get that $2 + 4n^3$ is $\Theta(n^3)$, which in the order of growth hierarchy Table 1.27 is strictly above $\lg n$.

- (C) True. As discussed in the section, all logarithmic functions grow at the same rate. So $\ln n$ is $\Theta(\lg n)$ and hence $\ln n$ is $\mathcal{O}(\lg n)$.
- (D) True by Theorem 1.15. L'Hôpital's Rule gives this

$$\lim_{x \rightarrow \infty} \frac{x^3 + x^2 + x}{x^3} = \lim_{x \rightarrow \infty} \frac{3x^2 + 2x + 1}{3x^2} = \lim_{x \rightarrow \infty} \frac{6x + 2}{6x} = \lim_{x \rightarrow \infty} \frac{6}{6} = 1$$

(these are a sequence of “ ∞/∞ ” limits).

(E) True. For any polynomial function p , Lemma 1.23 gives that p is $\mathcal{O}(2^n)$.

- V **V.1.42** In doing these we may have the option of using the algebraic properties in Lemma 1.25 to simplify and then refer to Table 1.27, or instead of using Theorem 1.15.

The former gives more of a sense of using how the expressions are constructed. For instance, in the first item below we strictly speaking do not need the first paragraph because the second paragraph suffices to show $k = 4$ is correct. But sticking to only the second paragraph could leave someone who had trouble with the question less sense of how they could produce $k = 4$ for themselves.

- (A) The smallest power is $k = 4$. By the lemma's item A the function $(1/10\,000\,000)n^4$ is $\mathcal{O}(n^4)$ and of course the function n^3 is $\mathcal{O}(n^3)$. By Table 1.27, n^3 is $\mathcal{O}(n^4)$. Thus by item B of the lemma, the function $n^3 + (1/10\,000\,000)n^4$ is $\mathcal{O}(n^4)$.

One way to see that no power $k < 4$ will do is to consider $\lim_{x \rightarrow \infty} x^3 / (x^3 + (x^4/10\,000\,000))$.

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{x^4}{x^3 + (x^4/10\,000\,000)} &= \lim_{x \rightarrow \infty} \frac{x}{1 + (x/10\,000\,000)} = \lim_{x \rightarrow \infty} \frac{x}{(10\,000\,000 + x)/10\,000\,000} \\ &= \lim_{x \rightarrow \infty} \frac{10\,000\,000x}{10\,000\,000 + x} = \lim_{x \rightarrow \infty} \frac{10\,000\,000}{1} = 10\,000\,000 \end{aligned}$$

(the next to last equality is from L'Hôpital's Rule). So $n^3 + (n^4/10\,000\,000)$ is $\Theta(n^4)$. Thus $k = 4$ is the minimum.

- (B) The smallest power is $k = 4$. Multiplying through gives $n^4 + 5n^3 + 6n^2 - n^2 \lg n - 5n \lg n - 6 \lg n$. Lemma 1.25's item B says that the entire expression is $\mathcal{O}(g)$ where g is the term with the largest order of growth. In this expression Table 1.27 shows that it is n^4 .

To check that no smaller power suffices, compare the functions with the limit.

$$\lim_{x \rightarrow \infty} \frac{x^4 + 5x^3 + 6x^2 - x^2 \lg x - 5x \lg x - 6 \lg x}{x^4} = \lim_{x \rightarrow \infty} \frac{x^4}{x^4} + \frac{5x^3}{x^4} + \frac{6x^2}{x^4} - \frac{x^2 \lg x}{x^4} - \frac{5x \lg x}{x^4} - \frac{6 \lg x}{x^4}$$

All of those limits are zero except for the first, which is 1. Therefore $x^4 + 5x^3 + 6x^2 - x^2 \lg x - 5x \lg x - 6 \lg x$ is $\Theta(x^4)$.

- (C) The answer is $k = 3$. The cosine function is bounded by 1. So Lemma 1.25 gives that the entire expression is $\mathcal{O}(n^3)$. We can use Theorem 1.15 to argue that $n = 3$ is right with this limit.

$$\lim_{x \rightarrow \infty} \frac{5x^3 + 25 + \lceil \cos(x) \rceil}{x^3} = \lim_{x \rightarrow \infty} \frac{5x^3}{x^3} + \frac{25 + \lceil \cos(x) \rceil}{x^3} = 5$$

The limit comes to a number intermediate between zero and infinity so $5x^3$ is $\Theta(x^3)$.

- (D) This answer is $k = 12$. Expanding the expression gives a leading term of $9n^{12}$. Either Lemma 1.25 or Theorem 1.15 then gives the answer.

- (E) This is $\mathcal{O}(n^{7/2})$ but since the question specifies that $k \in \mathbb{N}$, the answer is $k = 4$.

V V.1.43

- (A) The answer is 'both'. Both of these functions are $\Theta(n^2)$. So f is $\Theta(g)$.
 (B) The answer is that g is $\mathcal{O}(f)$ but not vice versa. Here, g is $\Theta(\lg n)$, while f is $\Theta(n^3)$. The benchmark table gives that g is $\mathcal{O}(f)$ but it is not the case that f is $\mathcal{O}(g)$.
 (C) The function f is quadratic while g is a square root. Thus g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.
 (D) The function f is $\mathcal{O}(n^{1.2})$ while the function g is $\mathcal{O}(n^{\sqrt{2}})$, which is approximately $\mathcal{O}(n^{1.414})$. We have that f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

V V.1.44 We simplify Big $\mathcal{O}$ expressions using Lemma 1.25.

- (A) Lemma 1.23 says that the exponential function $2^{(1/6)n}$ grows faster than the polynomial function n^6 . So f is $\mathcal{O}(g)$ but it is not the case the g is $\mathcal{O}(f)$.
 (B) Table 1.27 shows that the function 2^n is $\mathcal{O}(3^n)$, while 3^n is not $\mathcal{O}(2^n)$. Thus, g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.
 (C) Recall the logarithm rule that $\lg(3n) = \lg(3) + \lg(n)$. Because $\lg(3)$ is $\Theta(1)$, Lemma 1.25 says that $\lg(3n)$ is $\Theta(\lg n)$. From this it follows that the two have equivalent growth rates, so f is $\Theta(g)$.

V V.1.45 We will apply the definition, Definition 1.6, of Big $\mathcal{O}$, to show that Z is $\mathcal{O}(g)$ for any complexity function g .

Because g is a complexity function there is an $N \in \mathbb{N}$ such that if $x \geq N$ then $g(x)$ is defined and nonnegative. Take $C = 1$. Then for $x_0 > N$ we have that $|Z(x_0)| = 0 \leq C \cdot |g(x_0)| = C \cdot g(x_0)$.

V V.1.46

	$n = 1$	$n = 10$	$n = 100$	$n = 1000$
$\lg n$	–	1.05×10^{-17}	2.10×10^{-17}	3.15×10^{-17}
n	3.16×10^{-18}	3.16×10^{-17}	3.16×10^{-16}	3.16×10^{-15}
$n \lg n$	–	1.05×10^{-16}	2.10×10^{-15}	3.15×10^{-14}
n^2	3.16×10^{-18}	3.16×10^{-16}	3.16×10^{-14}	3.16×10^{-12}
n^3	3.16×10^{-18}	3.16×10^{-15}	3.16×10^{-12}	3.16×10^{-9}
2^n	6.33×10^{-18}	3.24×10^{-15}	4.01×10^{12}	3.39×10^{283}

V V.1.47 The number of seconds in a year is $60 \cdot 60 \cdot 24 \cdot 365.25 = 31\,557\,600$. At $10\,000 \times 10^6$ ticks per second, that is about 3.16×10^{17} ticks in a year (as also described in Table 1.29).

- (A) Solving $\lg(n) = 3.16 \times 10^{17}$ gives $n = 2^{3.16 \times 10^{17}} = 4.60 \times 10^{94\,997\,841\,911\,656\,529}$.
 (B) $(3.16 \times 10^{17})^2 = 9.96 \times 10^{34}$
 (C) 3.16×10^{17}
 (D) $(3.16 \times 10^{17})^{1/2} = 5.62 \times 10^8$

(E) Because $(3.16 \times 10^{11})^{1/3} \approx 680823.684681371$, the answer is 680 823.

(F) We have $\lg(3.16 \times 10^{17}) \approx 58.131$ so the answer is 58.

V **V.1.48** Solving $100\,000 \cdot n^2 = n^3$ gives $n = 100\,000$. So the first number n where $f(n) = 100\,000 \cdot n^2$ is less than $g(n) = n^3$ is $n = 100\,001$.

V **V.1.49** Linear. Finite State machines consume one character at each step. So they run in time equal to the length of the input. In the notation of Definition 1.30, $t_M(\sigma) = |\sigma|$.

V **V.1.50** Let g be the function $g(x) = 1$.

(A) We can directly apply the definition of Big $\mathcal{O}$, Definition 1.6. Take $N = 1$ and $C = 8$. If $x > N = 1$ then $C \cdot |g(x)| = 8 \cdot 1$ while $|f(x)| = 7$, so $|f(x)| \leq C|g(x)|$.

(B) The sine function has $-1 \leq \sin(x) \leq 1$ for all x . Take $N = 1$ and $C = 9$. If $x > N$ then $C \cdot |g(x)| = 9$ while $|f(x)| = 7 + \sin(x) \leq 8$.

(C) Fix $N = 1$ and $C = 8$. If $x \geq N = 1$ then $C \cdot |g(x)| = 8$ is greater than or equal to $|f(x)| = 7 + (1/x)$.

(D) We first show that if it is bounded then it is $\mathcal{O}(1)$. Suppose that f is bounded by $K \in \mathbb{R}$. If it is bounded by a smaller number then it is bounded by a larger so we can take $K \in \mathbb{R}^+$. Fix $N = 1$ and $C = K$. If $x \geq N = 1$ then $C \cdot |g(x)| = K \geq |f(x)|$, so f is $\mathcal{O}(g)$.

For the converse, suppose that f is $\mathcal{O}(g)$. Then there exists constants $N, C \in \mathbb{R}^+$ so that $x \geq N$ implies that $C \cdot |g(x)| = C \geq |f(x)|$. Choose $L = \max(C, f(0), \dots, f(N-1))$. Then for any $x \in \mathbb{N}$ we have $f(x) \leq L$.

V **V.1.51** By the last item in the prior question, a function $f: \mathbb{R} \rightarrow \mathbb{R}$ is $\mathcal{O}(1)$ iff it is bounded by a constant. So $g(x) \leq x^{\mathcal{O}(1)}$ says that the function g has at most a polynomial growth rate.

V **V.1.52** Let $N = 4$ and $C = 1$. Note that $4! = 24 > 2^4 = 16$. With that, if $n \geq N = 4$ then $C \cdot n! = n! = 4! \cdot 5 \cdot 6 \cdots (n-1) \cdot n \geq 2^4 \cdot 2^{n-4} = 2^n$.

V **V.1.53**

(A) This limit equals 0

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^2}{x^d} = \lim_{x \rightarrow \infty} \frac{2 \log_b(x) \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{2}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{\log_b(x)}{x^d}$$

by the calculation in Example 1.20.

(B) This limit equals 0

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^3}{x^d} = \lim_{x \rightarrow \infty} \frac{3(\log_b(x))^2 \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{3}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{(\log_b(x))^2}{x^d}$$

by the calculation in the prior item.

(C) The prior item suggests an argument by induction. So assume that the statement holds for the powers $n = 1, n = 2, \dots, n = k$ and consider the $n = k + 1$ case.

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^{k+1}}{x^d} = \lim_{x \rightarrow \infty} \frac{(k+1)(\log_b(x))^k \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{k+1}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{(\log_b(x))^k}{x^d}$$

Finish by applying the $n = k$ inductive hypothesis.

V **V.1.54** The function (which is uncomputable)

$$K(x) = \begin{cases} 0 & \text{if } \phi_x(x) \uparrow \\ 1 & \text{if } \phi_x(x) \downarrow \end{cases}$$

is bounded and so is $\mathcal{O}(1)$.

V **V.1.55** Use Theorem 1.15.

$$\lim_{x \rightarrow \infty} \frac{2^x}{3^x} = \lim_{x \rightarrow \infty} \left(\frac{2}{3} \right)^x = 0$$

For the second, $3^x = 2^{\lg(3) \cdot x}$ where $\lg(3)$ is the base 2 logarithm of 3.

V **V.1.56** Consider the ratio.

$$\frac{n!}{n^n} = \frac{1}{n} \cdot \frac{2}{n} \cdots \frac{n}{n} \leq \frac{1}{n} \cdot 1 \cdots 1$$

Clearly the limit of the ratio $n!/n^n$ is 0.

V **V.1.57** It is (D).

To establish $\mathcal{O}(f_2) < \mathcal{O}(f_3)$ we have

$$\lim_{x \rightarrow \infty} \frac{x^{\ln x}}{x^{\sqrt{x}}} = \lim_{x \rightarrow \infty} x^{\ln x - \sqrt{x}} = 0$$

because $\lim_{x \rightarrow \infty} (\ln x - \sqrt{x}) = -\infty$. (One way to prove this limit is to use Example 1.20. Another way is to multiply $\ln x - \sqrt{x}$ by the fraction $(\ln x + \sqrt{x})/(\ln x + \sqrt{x})$. That gives this.

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{(\ln x)^2 - x}{\sqrt{x} + \ln(x)} &= \lim_{x \rightarrow \infty} \frac{2 \ln x \cdot (1/x) - 1}{1/(2x^{1/2}) + 1/x} = \lim_{x \rightarrow \infty} \frac{2x}{2x} \cdot \frac{2 \ln x \cdot (1/x) - 1}{1/(2x^{1/2}) + 1/x} = 2 \cdot \lim_{x \rightarrow \infty} \frac{2 \ln x - x}{\sqrt{x} + 2} \\ &= 2 \cdot \lim_{x \rightarrow \infty} \frac{(2/x) - 1}{1/(2x^{1/2})} = 4 \cdot \lim_{x \rightarrow \infty} x^{1/2} \cdot \left(\frac{2}{x} - 1\right) = 4 \cdot \lim_{x \rightarrow \infty} 2x^{-1/2} - x^{1/2} = 4 \cdot \lim_{x \rightarrow \infty} \frac{2 - x}{\sqrt{x}} \\ &= 4 \cdot \lim_{x \rightarrow \infty} \frac{-1}{1/(2x^{1/2})} = 4 \cdot \lim_{x \rightarrow \infty} -2\sqrt{x} \end{aligned}$$

That's $-\infty$.)

For $\mathcal{O}(f_3) < \mathcal{O}(f_1)$ take the limit of the natural logarithm of the numerator and denominator. Since the natural logarithm is a strictly increasing function, this suffices for the result.

$$\lim_{x \rightarrow \infty} \frac{\ln(f_3(x))}{\ln(f_1(x))} = \lim_{x \rightarrow \infty} \frac{\ln(x^{\sqrt{x}})}{\ln(10^x)} = \lim_{x \rightarrow \infty} \frac{\sqrt{x} \ln(x)}{x \ln(10)} = \frac{1}{\ln(10)} \cdot \lim_{x \rightarrow \infty} \frac{\ln(x)}{\sqrt{x}}$$

Here also Example 1.20 applies, or note that this is an “ ∞/∞ ” type limit and apply L'Hôpital.

$$\lim_{x \rightarrow \infty} \frac{1/x}{1/(2x^{1/2})} = 2 \cdot \lim_{x \rightarrow \infty} \frac{\sqrt{x}}{x} = 2 \cdot \lim_{x \rightarrow \infty} \frac{1}{\sqrt{x}} = 0$$

V **V.1.58**

(A) Apply L'Hôpital's Rule twice, after verifying that they are both “ ∞/∞ ” limits.

$$\lim_{x \rightarrow \infty} \frac{x^2 + 5x + 1}{x^2} = \lim_{x \rightarrow \infty} \frac{2x + 5}{2x} = \lim_{x \rightarrow \infty} \frac{2}{2} = 1$$

(B) Apply L'Hôpital's Rule also here, remembering to use the Chain Rule for the numerator.

$$\lim_{x \rightarrow \infty} \frac{\lg(x+1)}{\lg(x)} = \lim_{x \rightarrow \infty} \frac{(1/x \ln(2)) \cdot 1}{1/x \ln(2)} = 1$$

V **V.1.59** No. If $\mathcal{O}(f)$ is the set of polynomials then, because it is a member of that set, f must be a polynomial. However, then f has some degree and no polynomial of higher degree is a member of $\mathcal{O}(f)$.

V **V.1.60** We have $x^{\lg x} = (2^{\lg x})^{\lg x} = 2^{(\lg x)^2} \in \mathcal{O}(2^{\text{poly}(\lg x)})$ (the first equality comes from the identity $x = 2^{\lg x}$).

V **V.1.61** Recall the logarithm rules that $\log_b(x^a) = a \cdot \log_b x$ and $b^{\log_b(x)} = x$.

(A) Fix a constant $k \in \mathbb{R}$. Consider the ratio.

$$\lim_{x \rightarrow \infty} \frac{k \lg x}{(\lg x)^2} = k \cdot \lim_{x \rightarrow \infty} \frac{1}{\lg x} = 0$$

Theorem 1.15 gives the desired conclusion.

(B) By Theorem 1.15, it suffices to show that this is 0.

$$\lim_{x \rightarrow \infty} \frac{x^k}{x^{\lg(x)}} = \lim_{x \rightarrow \infty} x^{k - \lg(x)}$$

Rewrite using the identity that $a = 2^{\lg(a)}$.

$$= \lim_{x \rightarrow \infty} 2^{\lg(x^{k - \lg(x)})} = \lim_{x \rightarrow \infty} 2^{(k - \lg(x)) \cdot \lg(x)} = \lim_{x \rightarrow \infty} 2^{k \lg(x) - (\lg(x))^2}$$

The prior item gives that the limit of the exponent is $-\infty$, and so the limit of the entire expression is 0.

(c) L'Hôpital's Rule gives this.

$$\lim_{x \rightarrow \infty} \frac{(\lg x)^2}{x} = \lim_{x \rightarrow \infty} \frac{2 \lg x \cdot (1/x)}{1} = 2 \lim_{x \rightarrow \infty} \frac{\lg x}{x} = 2 \lim_{x \rightarrow \infty} \frac{(1/x)}{1} = 2 \lim_{x \rightarrow \infty} \frac{1}{x} = 0$$

(D) Apply Theorem 1.15 along with L'Hôpital's Rule

$$\lim_{x \rightarrow \infty} \frac{x^{\lg x}}{2^x} = \lim_{x \rightarrow \infty} \frac{(2^{\lg x})^{\lg x}}{2^x} = \lim_{x \rightarrow \infty} \frac{2^{(\lg x)^2}}{2^x} = \lim_{x \rightarrow \infty} 2^{(\lg x)^2 - x}$$

(the first equality uses the identity $a = 2^{\lg(a)}$). The prior item gives that the limit of the exponent is $\lim_{x \rightarrow \infty} (\lg x)^2 - x = -\infty$, and so the limit of the entire expression is 0.

V 1.62 We will use the notation from the lemma statement.

(A) Suppose that $a \in \mathbb{R}$ is nonnegative. Since f is a complexity function there is an $N \in \mathbb{R}$ so that $x \geq N$ implies that f is defined and nonnegative. Fix $C = a$. If $x \geq N$ then $|af(x)| = af(x) \leq a \cdot f(x) = a|f(x)|$, as required.

(B) Suppose that f_1 is $\mathcal{O}(f_0)$. By definition there are constants $N, C \in \mathbb{R}$ so that if $x \geq N$ then $|f_1(x)| \leq C \cdot |f_0(x)|$. Because these two functions are complexity functions we can assume that N is so large that both are nonnegative for all inputs x . That gives $f_0(x) + f_1(x) \leq f_0(x) + Cf_0(x) = (1 + C) \cdot f_0(x)$, and hence there is a constant, $1 + C$, giving that $f_0 + f_1$ is in $\mathcal{O}(f_0)$.

The argument for $f_0 - f_1$ is the same except that the constant is $1 - C$.

(c) Take $f_0(x) = x^2$ and $f_1(x) = x$. Then the function $f_0 f_1$ is given by $f_0 f_1(x) = x^3$, so $f_0 f_1$ is not in $\mathcal{O}(f_0)$.

V 1.63

(A) We must show that f is in $\mathcal{O}(f)$. The function f is a complexity function, so there is some number beyond which it is defined and positive. Take N to be that number and take $C = 1$.

(B) Suppose that f_0 is $\mathcal{O}(f_1)$ and that f_1 is $\mathcal{O}(f_2)$. Then there exists $N_0, N_1, C_0, C_1 \in \mathbb{R}$ such that if $x \geq N_0$ then $C_0 \cdot f_1(x) \geq f_0(x)$, and if $x \geq N_1$ then $C_1 \cdot f_2(x) \geq f_1(x)$. Take $\hat{N} = \max(N_0, N_1)$ and $\hat{C} = C_0 \cdot C_1$. Then $x \geq \hat{N}$ implies that $\hat{C} \cdot f_2(x) = C_0 C_1 \cdot f_2(x) \geq C_0 \cdot f_1(x) \geq f_0(x)$.

V 1.64 Recall that by definition f is $\mathcal{O}(g)$ if and only if there exists numbers $N, C \in \mathbb{R}$ such that if $x \geq N$ then $|f(x)| \leq C \cdot |g(x)|$.

(A) If $\lim_{x \rightarrow \infty} f(x)/g(x)$ exists and equals 0 then for any $\varepsilon > 0$ there is an $M \in \mathbb{R}$ such that $x \geq M$ implies that $|f(x)/g(x)| < \varepsilon$, and therefore $|f(x)| < \varepsilon \cdot |g(x)|$. Taking N to be M and C to be ε , this is exactly what is required to conclude that f is $\mathcal{O}(g)$.

(B) Suppose otherwise, that g is $\mathcal{O}(f)$. Then by definition there are constants $K, P \in \mathbb{R}$ such that if $x \geq K$ then $|g(x)| \leq P \cdot |f(x)|$, which gives $1/P \leq |f(x)/g(x)|$.

The question's assumption is that $\lim_{x \rightarrow \infty} f(x)/g(x)$ exists and equals 0. Fix $\varepsilon = 1/(P + 1)$. Then there exists a constant $M \in \mathbb{R}$ so that if $x \geq M$ then $|f(x)/g(x)| \leq 1/(P + 1)$.

Those two paragraphs conflict for x 's larger than the maximum of K and M .

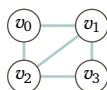
V 1.65 First use L'Hôpital's Rule to get that if $g(x) = a_m x^m + \dots + a_0$ then g is $\Theta(x^m)$. Then Example 1.20 shows that any logarithmic function is $\mathcal{O}(g)$.

For the other half, fix a base $b > 1$ and consider the exponential function $h(x) = b^x$. L'Hôpital's Rule gives this

$$\lim_{x \rightarrow \infty} \frac{x^m}{b^x} = \frac{m}{\ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{x^{m-1}}{b^x} = \frac{m(m-1)}{(\ln(b))^2} \cdot \lim_{x \rightarrow \infty} \frac{x^{m-2}}{b^x} = \dots = \frac{m!}{(\ln(b))^m} \cdot \lim_{x \rightarrow \infty} \frac{1}{b^x} = 0$$

and Theorem 1.15 gives the desired conclusion.

- V **V.2.41** For the octahedron, the path $\langle v_0, v_1, v_5, v_4, v_3, v_2, v_0 \rangle$ will do. For the icosahedron, one path is $\langle v_0, v_{10}, v_{11}, v_8, v_9, v_6, v_7, v_2, v_3, v_4, v_5, v_1, v_0 \rangle$.
- V **V.2.42** The same as the number of vertices. Traversing the Hamiltonian path means that there is one and only one edge leaving each vertex. So if a graph has n vertices then a Hamiltonian path has the same number of edges, n .
- V **V.2.43**
- (A) This graph has a Hamiltonian circuit, around the outside.



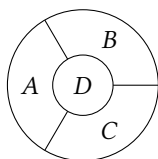
To include the vertices v_0 and v_3 , any circuit must include those four outside edges. In every Hamiltonian circuit the number of edges equals the number of vertices and hence no Hamiltonian circuit can contain the edge v_1v_2 , because that would make five edges.

- (B) Assume that all the edge weights are positive (if the given graph does not have that then to all weights, add a number large enough to make them all positive). To the edge e assign this weight.

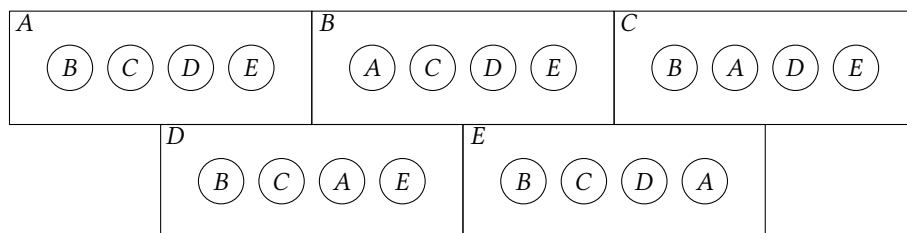
$$-(\text{number of vertices}) \cdot (\text{max weight}) - 1$$

Every Hamiltonian path has the same number of edges, specifically the number of edges equals the number of vertices. So a path through e has a negative total weight, and this is clearly the only way to get a negative total weight, so the Hamiltonian path solving the Traveling Salesman must include e .

- V **V.2.44** We can do these by eye.
- (A) The shortest is q_2 to q_1 to q_7 , which has length 14.
- (B) The shortest is q_0 to q_3 to q_4 to q_8 , of length 23.
- (C) There is no path between these two vertices.
- V **V.2.45** This is the natural one. Every region borders all three others so three colors does not suffice.

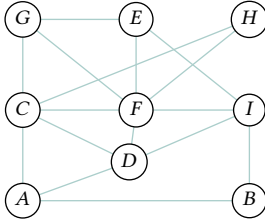


- V **V.2.46**
- (A) This map shows five countries. They are not contiguous; for instance, country A has five separate components (the first component contains B, C, D, and E).



Because of the portion in the upper left, country A must be a different color than any of the others. The other portions similarly give that all the other countries are different colors than each other. In total there are five countries, each a different color than all the others.

- (B) Make a circle divided into five wedges.
- V **V.2.47** For reference here is the graph.

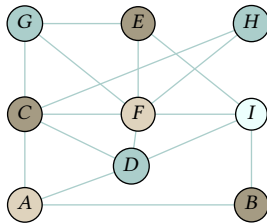


- (A) Suppose that a three-coloring suffices. The three vertices A , C , and D form a triangle so they must be different colors. Since CDF also is a triangle, we conclude that A and F are the same colors. So far we have: $C_0 = \{A, F\}$, $C_1 = \{C\}$, $C_2 = \{D\}$.

Next, consider triangle DFI . Its presence gives that C and I are the same colors. So we have $C_0 = \{A, F\}$, $C_1 = \{C, I\}$, $C_2 = \{D\}$. Then triangle EFI leads to $C_0 = \{A, F\}$, $C_1 = \{C, I\}$, $C_2 = \{D, E\}$.

Finally, triangle CFG gives that D and G are the same color, giving $C_0 = \{A, F\}$, $C_1 = \{C, I\}$, $C_2 = \{D, E, G\}$. But that's impossible because G is connected to E .

- (B) This is a four-coloring.



$$\begin{aligned} C_1 &= \{A, F\} \\ C_2 &= \{B, C, E\} \\ C_3 &= \{D, G, H\} \\ C_4 &= \{I\} \end{aligned}$$

V V.2.48 A formula is satisfiable if there is a T in the truth table's final column.

- (A) The formula $(P \wedge Q) \vee (\neg Q \wedge R)$ is satisfiable.

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

- (B) The formula $(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$ is not satisfiable.

P	Q	$P \rightarrow Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg((P \wedge Q) \vee \neg P)$	$(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V V.2.49

- (A) There are four lines where the input causes the expression to evaluate to F .

Line	Clause
$F-T-F$	$P \vee \neg Q \vee R$
$F-T-T$	$P \vee \neg Q \vee \neg R$
$T-F-T$	$\neg P \vee Q \vee \neg R$
$T-T-T$	$\neg P \vee \neg Q \vee \neg R$

- (B) Any clause where the elements are joined by $\vee$'s will evaluate to T unless every single element is F . These clauses are constructed to make every element false on the targeted line. On any other line at least one element is true.

- (C) The Conjunctive Normal form is this.

$$(P \vee \neg Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee \neg R) \wedge (\neg P \vee \neg Q \vee \neg R)$$

This statement evaluates to F if and only if any of its constituent clauses is F . By the prior item, that happens if and only if the input is a targeted line.

- V **V.2.50** It does not imply that. For an example, let the Boolean expression be $P \vee Q$.

P	Q	$P \vee Q$	$\neg(P \vee Q)$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	T	F

It is satisfiable by taking both variables to be T . Its negation is also satisfiable, by taking both variables to be F

- V **V.2.51** A finite set of propositional logic statements $\{S_0, \dots, S_{k-1}\}$ is satisfiable if and only if the single statement $S_0 \wedge S_1 \dots \wedge S_{k-1}$ is satisfiable.

- V **V.2.52**

- (A) AT&T connects to Citigroup, which connects to Ford Motor.
 (B) Haliburton connects to Citigroup, which connects to Ford Motor.
 (C) There is no connection between Caterpillar and Ford Motor.
 (D) Also no connection.

- V **V.2.53** (A) 2 (B) 2 (C) 0 (D) 4 (I was only in deleted scenes. But all this is just for fun.)

- V **V.2.54**

- (A) A vertex cover with $B = 2$ elements is $S = \{v_1, v_3\}$. An independent set with $\hat{B} = 4$ elements is $\hat{S} = \{q_0, q_2, q_4, q_5\}$.
 (B) A vertex cover with $B = 3$ elements is $S = \{v_1, v_2, v_4\}$. An independent set with $\hat{B} = 3$ elements is $\hat{S} = \{v_0, v_3, v_5\}$.
 (C) A vertex cover with $B = 4$ elements is $S = \{v_2, v_5, v_8, v_9\}$. An independent set with $\hat{B} = 6$ elements is $\hat{S} = \{v_0, v_1, v_3, v_4, v_6, v_7\}$.
 (D) For any edge, if it has at least one endpoint in S then it has at most one endpoint in the complement $\hat{S} = \mathcal{N} - S$. Conversely, if an edge has at most one endpoint in $\hat{S}$ then it has at least one endpoint in the complement $S = \mathcal{N} - \hat{S}$.

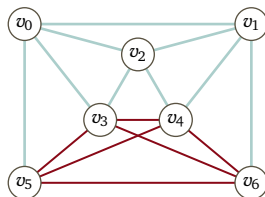
- V **V.2.55** A 3-clique is a triangle. A 2-clique is an edge.

- V **V.2.56** A k -clique has $k(k-1)/2$ edges, one for each combination of two vertices.

V.2.57 We could list all triples of vertices and then check each to see whether it is in the graph. But instead this list is by eye.

$$\{v_0, v_1, v_2\}, \{v_0, v_1, v_6\}, \{v_0, v_2, v_6\}, \{v_2, v_3, v_4\}, \{v_2, v_3, v_5\}, \{v_2, v_4, v_5\}, \{v_3, v_4, v_5\}$$

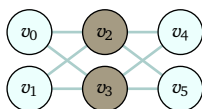
- V **V.2.58** It has a 4-clique, and thus it necessarily has a 3-clique.



But it has no 5-clique, as checked by exhaustion in Figure 1 on page 13.

- V **V.2.59** The one on the right does not partition the vertices into two disjoint sets, it partitions into five sets: $\{v_0\}$, $\{v_1\}$, $\{v_2, v_3\}$, $\{v_4\}$, and $\{v_5\}$.

- V **V.2.60** The graph cut $\{\{v_0, v_1, v_4, v_5\}, \{v_2, v_3\}\}$



Combination of five vertices					A missing edge	Combination of five vertices					A missing edge
v_0	v_1	v_2	v_3	v_4	v_1v_3	v_0	v_2	v_3	v_4	v_6	v_0v_4
v_0	v_1	v_2	v_3	v_5	v_2v_5	v_0	v_2	v_3	v_5	v_6	v_0v_6
v_0	v_1	v_2	v_3	v_6	v_2v_6	v_0	v_2	v_4	v_5	v_6	v_0v_4
v_0	v_1	v_2	v_4	v_5	v_0v_4	v_0	v_3	v_4	v_5	v_6	v_0v_4
v_0	v_1	v_2	v_4	v_6	v_0v_6	v_1	v_2	v_3	v_4	v_5	v_1v_3
v_0	v_1	v_2	v_5	v_6	v_0v_6	v_1	v_2	v_3	v_4	v_6	v_1v_3
v_0	v_1	v_3	v_4	v_5	v_0v_4	v_1	v_2	v_3	v_5	v_6	v_1v_3
v_0	v_1	v_3	v_4	v_6	v_0v_6	v_1	v_2	v_4	v_5	v_6	v_1v_5
v_0	v_1	v_3	v_5	v_6	v_0v_6	v_1	v_3	v_4	v_5	v_6	v_1v_3
v_0	v_1	v_4	v_5	v_6	v_0v_6	v_2	v_3	v_4	v_5	v_6	v_2v_5
v_0	v_2	v_3	v_4	v_5	v_0v_4						

FIGURE 1, FOR QUESTION V.2.58: A check of all possible 5-cliques.

gives a cut set with eight elements, $\{v_0v_2, v_0v_3, v_1v_2, v_1v_3, v_2v_5, v_3v_4, v_3v_5\}$. This graph cut is maximal because its cut set contains all of the edges.

- V **V.2.61** The definition of a **Three-dimensional Matching** problem requires three sets, here the instructors, the courses, and the time slots, and that they all have the same size, here five. We must be given as input a set $M \subseteq X \times Y \times Z$, and here that set consists of all triples that the graph shows as a possibility, so $M = \{(A, 1, \beta), (A, 1, \epsilon), (A, 2, \gamma), \dots\}$. We must decide if there is a set $\hat{M} \subseteq M$ containing n elements (here, five elements) such that no two of the triples in $\hat{M}$ agree on their first coordinates, or their second or third coordinates either. That fits this problem because we cannot have the same instructor teaching two sections, or two instructors teaching the same section, or two sections using the room during the same time slot.

A suitable matching is $\hat{M} = \{(A, 2, \epsilon), (B, 0, \alpha), (C, 3, \gamma), (D, 1, \beta), (E, 4, \delta)\}$.

V **V.2.62**

- (A) The set $M = X \times Y \times Z$ contains these twenty seven triples $\langle x, y, z \rangle$.

$z = a$:			$z = d$:			$z = e$:		
$\langle a, b, a \rangle$	$\langle a, c, a \rangle$	$\langle a, d, a \rangle$	$\langle a, b, d \rangle$	$\langle a, c, d \rangle$	$\langle a, d, d \rangle$	$\langle a, b, e \rangle$	$\langle a, c, e \rangle$	$\langle a, d, e \rangle$
$\langle b, b, a \rangle$	$\langle b, c, a \rangle$	$\langle b, d, a \rangle$	$\langle b, b, d \rangle$	$\langle b, c, d \rangle$	$\langle b, d, d \rangle$	$\langle b, b, e \rangle$	$\langle b, c, e \rangle$	$\langle b, d, e \rangle$
$\langle c, b, a \rangle$	$\langle c, c, a \rangle$	$\langle c, d, a \rangle$	$\langle c, b, d \rangle$	$\langle c, c, d \rangle$	$\langle c, d, d \rangle$	$\langle c, b, e \rangle$	$\langle c, c, e \rangle$	$\langle c, d, e \rangle$

- (B) One correct answer is $\hat{M} = \{\langle a, b, a \rangle, \langle b, c, d \rangle, \langle c, d, e \rangle\}$.

V.2.63 Figure 2 on page 14 is a check of every possibility.

- V **V.2.64** Figure 3 on page 14 shows the result of writing a small script.
- V **V.2.65** There are a total of 538 electoral votes. A 269-269 tie is easy to concoct. For example, the electoral votes of CA, TX, FL, NY, IL, PA, OH, GA, MI, NC, VA, and VT add to 269.
- V **V.2.66** There are twenty five: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, and 97.
- V **V.2.67**
- (A) Yes, it is prime.
- (B) No, it is not prime. Besides the trivial ones, the divisors of 6 165 are 3, 5, 9, 15, 45, 137, 411, 685, 1 233, and 2 055.
- (C) Yes.
- (D) No. Besides itself and 1, this number is also divisible by 7 and 601.
- (E) No. Besides the trivial divisors of itself and 1, this number also has the divisors 3, 11, 33, 233, 699, and 2 563.
- V **V.2.68**
- (A) One is 3. The list of nontrivial divisors is 3, 9, 3 469, and 10 407.
- (B) One is 2. The list of nontrivial ones is 2, 4, 8, 6 553, 13 106, and 26 212.
- (C) One is 2. The list of nontrivial divisors is 2, 3, 4, 5, 6, 8, 10, 12, 15, 16, 20, 24, 25, 30, 32, 40, 48, 50,

21	33	49	42	19	Total	21	33	49	42	19	Total
N	N	N	N	N	0	Y	N	N	N	N	21
N	N	N	N	Y	19	Y	N	N	N	Y	40
N	N	N	Y	N	42	Y	N	N	Y	N	63
N	N	N	Y	Y	61	Y	N	N	Y	Y	82
N	N	Y	N	N	49	Y	N	Y	N	N	70
N	N	Y	N	Y	68	Y	N	Y	N	Y	89
N	N	Y	Y	N	91	Y	N	Y	Y	N	112
N	N	Y	Y	Y	110	Y	N	Y	Y	Y	131
N	Y	N	N	N	33	Y	Y	N	N	N	54
N	Y	N	N	Y	52	Y	Y	N	N	Y	73
N	Y	N	Y	N	75	Y	Y	N	Y	N	96
N	Y	N	Y	Y	94	Y	Y	N	Y	Y	115
N	Y	Y	N	N	82	Y	Y	Y	N	N	103
N	Y	Y	N	Y	101	Y	Y	Y	N	Y	122
N	Y	Y	Y	N	124	Y	Y	Y	Y	N	145
N	Y	Y	Y	Y	143	Y	Y	Y	Y	Y	164

FIGURE 2, FOR QUESTION V.2.63: All possibilities in the Subset Sum instance.

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	Weight	Value	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	Weight	Value
N	N	N	N	N	0	0	Y	N	N	N	N	21	50
N	N	N	N	Y	19	40	Y	N	N	N	Y	40	90
N	N	N	Y	N	42	44	Y	N	N	Y	N	63	94
N	N	N	Y	Y	61	84	Y	N	N	Y	Y	82	134
N	N	Y	N	N	49	34	Y	N	Y	N	N	70	84
N	N	Y	N	Y	68	74	Y	N	Y	N	Y	89	124
N	N	Y	Y	N	91	78	Y	N	Y	Y	N	112	128
N	N	Y	Y	Y	110	118	Y	N	Y	Y	Y	131	168
N	Y	N	N	N	33	48	Y	Y	N	N	N	54	98
N	Y	N	N	Y	52	88	Y	Y	N	N	Y	73	138
N	Y	N	Y	N	75	92	Y	Y	N	Y	N	96	142
N	Y	N	Y	Y	94	132	Y	Y	N	Y	Y	115	182
N	Y	Y	N	N	82	82	Y	Y	Y	N	N	103	132
N	Y	Y	N	Y	101	122	Y	Y	Y	N	Y	122	172
N	Y	Y	Y	N	124	126	Y	Y	Y	Y	N	145	176
N	Y	Y	Y	Y	143	166	Y	Y	Y	Y	Y	164	216

FIGURE 3, FOR QUESTION V.2.64: Output of a brute force script to show that there is no solution of an instance of the Knapsack problem.

Year	Winner	Other	Tallest?	Year	Winner	Other	Tallest?
1900	W McKinley	W J Bryan	N	1964	L B Johnson	B Goldwater	Y
1904	T Roosevelt	A B Parker	Y	1968	R Nixon	H Humphrey	Y
1908	W H Taft	W J Bryan	Y	1972	R Nixon	G McGovern	N
1912	W Wilson	-2 others-	N	1976	J Carter	G Ford	N
1916	W Wilson	C E Hughes	Y	1980	R Reagan	J Carter	Y
1920	W G Harding	J M Cox	Y	1984	R Reagan	W Mondale	Y
1924	C Coolidge	J W Davis	N	1990	G H W Bush	M Dukakis	Y
1928	H Hoover	A Smith	Y	1992	W Clinton	G H W Bush	-same-
1932	F D Roosevelt	H Hoover	Y	1996	W Clinton	R Dole	Y
1936	F D Roosevelt	A Landon	Y	2000	G W Bush	A Gore	N
1940	F D Roosevelt	W Willkie	N	2004	G W Bush	J Kerry	N
1944	F D Roosevelt	T Dewey	Y	2008	B Obama	J McCain	Y
1948	H S Truman	T Dewey	Y	2012	B Obama	M Romney	N
1952	D Eisenhower	A Stevenson II	Y	2016	D Trump	H Clinton	Y
1956	D Eisenhower	A Stevenson II	Y	2020	J Biden	D Trump	N
1960	J F Kennedy	R Nixon	Y	2024	D Trump	K Harris	Y

FIGURE 4, FOR QUESTION V.3.7: Height comparison of US Presidential winner and main opponent, since 1900.

60, 64, 75, 80, 96, 100, 120, 128, 150, 160, 192, 200, 240, 300, 320, 384, 400, 480, 600, 640, 800, 960, 1 200, 1 600, 1 920, 2 400, 3 200, and 4 800.

(D) One is 61. The list of nontrivial divisors is 61, 71.

(E) The only ones are 1 and 877. It is prime.

V **V.2.69** The AKS algorithm that solves Primality answers ‘yes’ or ‘no’ without finding divisors.

V **V.3.7** *Note:* the meanings give here are common usage. But because these terms are not formally defined, different authors may use them in slightly different ways.

(A) A heuristic is a shortcut, an approach to a problem that is more reliable than simple guessing and is faster than a full solution (or perhaps no full solution is possible). But the connotation of ‘heuristic’ is that it is not a theoretically correct full solution.

One example is approaching the Traveling Salesman problem by using a greedy algorithm, that is, always going next to the nearest not-yet-visited city. Another example is that antivirus programs use heuristics to try and determine if an executable is malicious, such as comparing the first few bytes to a list of known-bad ones.

A third example is based on the data in Figure 4 on page 15. In the contests for US President since 1900, the taller candidate has won 21 out of 31 times, for a sample proportion of 0.68 (not counting 1992). The chance of such a result by happenstance is less than four percent. You could decide to bet in the next election on the taller candidate. This isn’t logical, but it is a method and it is used in practice.

So, a heuristic is a method that, while it may be used in practice, is not guaranteed to be optimal, perfect, or even rational. In contrast, an algorithm for a problem is guaranteed to solve that problem.

(B) Pseudocode is a description of an algorithm. It gives a sketch or outline, usually at a quite high level. It is often laid out as though it were compilable code but usually it does not conform to any specific language’s syntax; for instance it typically does not include details such as declaration of variables.

This is pseudocode for the recursive depth-first traversal of a binary tree.

```
process_subtree(v)
  process_subtree(v's left child)
  process_vertex(v)
  process_subtree(v's right child)
```

Another example describes the most simple algorithm for checking whether a number is prime, by dividing it by each natural smaller than it but bigger than one.

```

is_prime(n)
  for 2 <= i < n
    if i divides n
      return False
  return True

```

We have seen many other examples in this book, often given as flowcharts.

So pseudocode is close to a specification of an algorithm. But in ordinary usage, an algorithm is completely specified so that an implementing programmer does not need to work out any significant details, while pseudocode may leave some details unspecified. (An analogy: an experienced cook may describe making Scalloped Potatoes as, “bake thin-sliced potatoes in a white sauce with breadcrumbs on top.” That is not complete in that it depends on the implementer’s understanding a number of details, but it is headed in the direction of a recipe.)

- (C) Of course, in the first section of the first chapter we saw a number of Turing machines, such as one that computes the predecessor function, another that computes the sum of two inputs, and a third that never halts.

A Turing machine is more like a program than an algorithm, in that it is an implementation of an algorithm. For instance, we could have the Bubble Sort algorithm in mind and then implement it as a Turing machine.

- (D) A flowchart is pseudocode given in graphical form. It describes an algorithm, typically at a high level (sometimes even higher than that usually used for pseudocode). We have seen a number of flowcharts in this book, including in this chapter. Flowcharts differ from algorithms in the same way that pseudocode does.
- (E) Source code differs from an algorithm in that it is an implementation of an algorithm (sometimes a number of algorithms are bundled together to make one source). Source is targeted at a specific compiler or interpreter, and perhaps at a specific hardware and software platform.

For example, there is source code in Racket that implements the algorithm to determine if an input natural number n is prime by trying to divide by all numbers less than n .

- (F) An executable is a binary file that can be run on an operating system. It is usually the result of compiling source code, and so is an implementation of an algorithm. Three examples are the executable that results from compiling (and linking) the “Hello World!” program in C, the executable for your browser, and the executable that typesets input $\text{\LaTeX}$ code.
- (G) A process is an instance of a program that has been loaded into a machine’s memory for execution. As such, it is an implementation of an algorithm (or perhaps a number of algorithms). Three examples are the result of calling the executable of the “Hello World!” program, of calling the executable for your browser, and of invoking the executable for $\text{\LaTeX}$.

Some processes, such as an operating system shell or a web server, are designed to run forever, while typically in computational complexity discussions the term ‘algorithm’ is used for something that performs some task and then halts.

V V.3.8 A solution is an algorithm that acts as the characteristic function of the string set.

V V.3.9 A decision problem is any one that requires a ‘yes’ or ‘no’ answer. A language decision problem is a specialization of that, to ‘yes’ or ‘no’ answers about membership in the given language. (However, we often take languages as precise descriptions of the problems that we are working on, and the two terms may sometimes be casually conflated.)

V V.3.10

(A) The multiplication is easy: $(1/0.01)^2 \cdot 21 \cdot 30 = 6.30 \times 10^6$.

(B) $1 + \lfloor \lg(1\,000\,000) \rfloor = 20$

(C) Multiply $(6.30 \times 10^6) \cdot 20 = 1.26 \times 10^8$. There are eight million bits in a megabyte. So the total is less than 16 megabytes, about eight books from Project Gutenberg or a ten minute phone video (in H.264 at 1080).

V.3.11 The output is not determined from the input. For instance, if the input query asks to produce a US state that contains a city named Springfield then one correct output is Alaska while another is Vermont. But a function must be well-defined, so the output is uniquely determined by the input.

V V.3.12 True, the requirement that an algorithm is well-specified means that two implementations must have

the same input/output behavior.

The converse does not hold; two programs computing the same function need not implement the same algorithm. For instance, two programs may both sort input strings but one uses Bubble Sort while the other uses Quicksort.

V V.3.13

- (A) Recognizing a correctly sorted string is a different task than finding a good way (or indeed the best way) to sort an unsorted string.
- (B) Similarly, recognizing a correct permutation feels different—and much easier—than generating that permutation when given an unsorted string.

V V.3.14 These algorithms signal that the input is a member of the language by returning 1, and signal that it is not by returning 0.

- (A) Because the number 4 is a square that is one greater than a prime, three members are $\langle 0, 4 \rangle$, $\langle 1, 3 \rangle$, and $\langle 2, 2 \rangle$. An algorithm computing the characteristic function is to input the sequence and unpack it to get $n, m \in \mathbb{N}$. Then add the two numbers. If the sum is both square and one greater than a prime then return 1, otherwise return 0.
- (B) Three string members of $\mathcal{L}$ are $\sigma_0 = 200$, $\sigma_1 = 100$, and $\sigma_2 = 300$. The algorithm is: if the input is a single 0 or ends in two 0's, then return 1. Otherwise return 0.
- (C) The language $\mathcal{L}_2$ contains 11, 110, and 1. A natural algorithm is to scan the input string, keeping a running count of the number of 0's and 1's. At the end if there are more 1's then return 1, otherwise return 0.
- (D) Recall that strings equal to their reverse are called palindromes. Three palindromes over $\mathbb{B}^*$ are 101, 11, and 1. To compute the characteristic function, read and save the input string. Reverse the string and compare it with the original. If it matches then return 1 and otherwise return 0.

V V.3.15 These algorithms return 1 if they accept the input as a member of the language, and 0 if they do not.

- (A) The empty language is $\mathcal{L} = \emptyset = \{ \}$. Its characteristic function is $\mathbb{1}_{\mathcal{L}}(n) = 0$. The algorithm to solve this language decision problem is to, for all input, return 0.
- (B) This language contains two strings, $\mathbb{B} = \{ 0, 1 \}$ (technically these are characters, not length one strings, but in this book we feel free to discount the difference if that is convenient). The characteristic function is this.

$$\mathbb{1}_{\mathbb{B}}(\sigma) = \begin{cases} 1 & \text{if } \sigma = 0 \text{ or } \sigma = 1 \\ 0 & \text{otherwise} \end{cases}$$

The algorithm for computing that function is obvious.

- (C) The language $\mathbb{B}^*$ contains all of the bitstrings. The algorithm that solves the decision problem for this language will, given a bitstring, return 1.

V V.3.16 These algorithms signal they accept the input as a member of the language by returning 1 and signal rejection with 0.

- (A) For the algorithm, fix a Finite State machine $\mathcal{M}$ that recognizes the language described by a^*ba^* . Read the input and simulate running $\mathcal{M}$ on that input. This will terminate in a finite number of steps. If $\mathcal{M}$ ends in an accepting state then return 1, otherwise return 0.

Stated more simply, the algorithm outputs 1 if the input consists entirely of a's, with a single b, and otherwise outputs 0.

- (B) The grammar defines the language $\mathcal{L} = \{ a^n b^m \mid n, m \in \mathbb{N} \}$. The algorithm is: input the string and if all of the b's come after all of the a's then return 1. Otherwise, return 0.

V V.3.17 We will just suggest the algorithm.

First we observe that there is an algorithm that, given a machine $\mathcal{M}$ and a state q in that machine, finds the set of states reachable from q . The algorithm proceeds in steps. Step 1 initializes by taking $R_0 = \{ q \}$. Step $i + 1$ begins with the set of states R_i that are reachable from q with i -many transitions or fewer. In that step, to make R_{i+1} start with R_i . Add to it any states reachable in one transition from a state in R_i . If there are no such new states, so that $R_{i+1} = R_i$ then the algorithm stops. This algorithm must eventually terminate because the number of states in the machine is finite.

- (A) The language is empty if and only if from the start state, no accepting state is reachable.

- (B) Given $\mathcal{M}$, obtain a new machine $\hat{\mathcal{M}}$ that recognizes the complement of $\mathcal{L}(\mathcal{M})$ by copying $\mathcal{M}$, except setting each accepting state of $\mathcal{M}$ to be non-accepting in $\hat{\mathcal{M}}$ and setting each non-accepting state of $\mathcal{M}$ to be accepting in $\hat{\mathcal{M}}$. Then $\mathcal{L}(\mathcal{M}) = \Sigma^*$ if and only if $\mathcal{L}(\hat{\mathcal{M}})$ is empty, and we have reduced to this exercise's first item.
- (C) The language is infinite if and only if there is a state $\hat{q}$ that can be reached from the initial state, and some accepting state can be reached from $\hat{q}$, and is such that $\hat{q}$ can be reached from itself by a nonempty string. (That is, it can be pumped.)

V **V.3.18** Here we just suggest each algorithm.

- (A) See if the input string starts with a 1 bit.
- (B) The number represented in decimal as 1000 when converted to binary is 1111101000, which is ten bits. So if the input is eleven bits then it is in the language. If the input is fewer than ten bits then it is not in the language. If the input is exactly ten bits then there are a fixed number of cases to do, such as checking whether the second bit is 0, but that doesn't change that the algorithm is $\mathcal{O}(1)$.

V **V.3.19** This is a search problem since we need only produce a single bridge.

V **V.3.20** It can be hard to tell a decision problem from a language decision problem. The difference is that the latter is stated in terms of a language.

- (A) This is a yes-or-no question, so it is a decision problem.
- (B) Function problem.
- (C) Decision problem.
- (D) Search problem.
- (E) Search problem.
- (F) Decision problem.
- (G) Language decision problem.

V **V.3.21** It can be hard to tell a decision problem from a language decision problem. The difference is that the latter is stated in terms of a language.

- (A) As stated in Problem 2.10 it is a decision problem.
- (B) Search problem.
- (C) Function problem.
- (D) Search problem.
- (E) Decision problem.

V **V.3.22**

- (A) Search problem.
- (B) Decision problem.
- (C) Search problem.
- (D) Optimization problem. (The differences between problem types can be fuzzy. A person could reason that there might be more than one such submatrix and call this a search problem.)

V **V.3.23** For each we must give a set of strings.

- (A) A language suited to machines with unary input is $\mathcal{L}_0 = \{1^n \mid n \in \mathbb{N} \text{ is a square}\}$. A language suited to more everyday input is $\mathcal{L}_1 = \{d \in \{0, \dots, 9\}^+ \mid d \text{ represents a perfect square in base 10}\}$. We use $\{0, \dots, 9\}^+$ to mean strings of at least one decimal digit.
- (B) One language is this set of string triples.

$$\{\langle x, y, z \rangle \in (\{0, \dots, 9\}^+)^3 \mid \text{interpreted as decimal representations, } x^2 + y^2 = z^2\}$$

We use $\{0, \dots, 9\}^+$ to mean strings of at least one decimal digit.

- (C) A language is $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents a graph } \mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle \text{ where the number of edges in } \mathcal{E} \text{ is even}\}$.
- (D) The language is $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents a pair } \langle \mathcal{G}, p \rangle \text{ where } p \text{ is a path with a repeated vertex}\}$.

V **V.3.24** For each we must give a set of strings.

- (A) The Graph Coloring problem is stated as a decision problem, so we just need to give a suitable language.

$$\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents } \langle \mathcal{G}, k \rangle, \text{ where } \mathcal{G} \text{ is } k\text{-colorable}\}$$

Often that language would be written in this way

$$\mathcal{L} = \{ \langle \mathcal{G}, k \rangle \mid \mathcal{G} \text{ is } k\text{-colorable} \}$$

although that description does not explicitly mention the string representation.

- (B) The version given in the text is a search problem. This is a language for the language decision problem version.

$$\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents a graph that has a circuit that traverses each edge exactly once} \}$$

- (C) The **Shortest Path** problem is given in the text as a function problem. This is a language for the language decision problem.

$$\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents } \langle \mathcal{G}, v_0, v_1, p \rangle, \text{ where } p \text{ is a minimal-length path between the vertices} \}$$

- V **V.3.25** A language is a set of strings. So for each of these the members of the set are technically bitstrings. Thus in the first answer we could write the set of bitstrings that represent decimal numbers, but it is common to not emphasize that detail.

- (A) $\{ n \in \mathbb{N} \mid \text{the factors of } n \text{ sum to more than } 2n \}$.
 (B) $\{ \langle \mathcal{P}, i \rangle \mid \text{machine } \mathcal{P} \text{ on input } i \text{ halts in less than ten steps} \}$
 (C) $\{ E \mid \text{the logic expression } E \text{ can be satisfied by three distinct assignments} \}$
 (D) $\{ \langle \mathcal{G}, B \rangle \mid \text{for all } v_0, v_1 \in \mathcal{G} \text{ there is a path of cost less than } B \}$

- V **V.3.26** It is about deciding membership in the language $\{ \langle \mathcal{P}_e, e \rangle \mid \mathcal{P}_e \text{ halts on input } e \}$. (To emphasize that a language is a set of strings we could write $\mathcal{L} = \{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents a pair } \langle \mathcal{P}_e, e \rangle \text{ where } \mathcal{P}_e \text{ halts on } e \}$.)

- V **V.3.27** Note that this exercise does not call explicitly for language decision problems. It calls for decision problems.

- (A) Given a triple $\langle n_0, n_1, p \rangle \in \mathbb{N}^3$, decide if $p = n_0 \cdot n_1$.
 (B) Given a triple $\langle \mathcal{G}, v_0, v_1 \rangle$, where $\mathcal{G}$ is a weighted graph and $v_0 \neq v_1$ are vertices in that graph, decide if v_1 is a vertex that is nearest to v_0 .

- V **V.3.28** This family

$$\mathcal{L}_B = \{ \langle \mathcal{G}, v_0, v_1 \rangle \mid \text{There is a path in } \mathcal{G} \text{ from } v_0 \text{ to } v_1 \text{ of total weight less than or equal to } B \}$$

is parametrized by the bound B .

- V **V.3.29**

- (A) We use $\mathcal{B}$ to denote a game board, a 4×4 array containing the numbers 1 through 14 along with a blank. Take $B \in \mathbb{N}$ as the parameter.

$$\mathcal{L}_B = \{ \mathcal{B} \mid \mathcal{B} \text{ can be solved in fewer than } B \text{ moves} \}$$

- (B) Take $B \in \mathbb{N}$ as a bound. Write $\mathcal{R}$ for a Rubik's cube arrangement, a sequence of six 3×3 arrays populated by nine members of the set $C_0, \dots$ nine members of the set C_5 .

$$\mathcal{L}_B = \{ \mathcal{R} \mid \mathcal{R} \text{ can be solved in fewer than } B \text{ moves} \}$$

- (C) Take $B \in \mathbb{R}^+ \cup \{0\}$. We can denote a way to finish a car while satisfying the constraints with a sequence whose elements are jobs $C = \langle j_0, \dots, j_k \rangle$. With every such sequence there is a total time $t(C)$.

$$\mathcal{L}_B = \{ C \mid t(C) \leq B \}$$

- V **V.3.30** A function version inputs a graph and outputs the set of every Hamiltonian circuit. A search version inputs a graph and outputs a Hamiltonian circuit, if one exists, and some nominal value such as $\emptyset$ if no circuit exists. An optimization version outputs a circuit that is best by some criterion. For instance, in a weighted graph it returns the circuit of least weight; this is the **Traveling Salesman** problem.

- V **V.3.31** Observe that the empty set of vertices is independent, as is a set with one vertex. So every graph has a subset of its vertices that are independent.

- (A) Because of the observation we can't reasonably say, "Given a graph, decide if it has an independent set." We can instead use a parameter B : given a graph, decide if it has an independent set with B vertices.
- (B) For $B \in \mathbb{N}^+$ Use the language $\mathcal{L}_B = \{\mathcal{G} \mid \mathcal{G} \text{ has } B \text{ vertices that do not share an edge}\}$.
- (C) Given a graph G , find an independent set with B -many vertices. (There may be more than one, or none, but if we find one then we are done.)
- (D) Given a graph, return all independent sets of size B , or the string None.
- (E) Given a graph, return an independent set that is maximal, that is, it is as large as possible for the graph.
- V **V.3.32** One is the task of finding the maximum integer in a nonempty list. The natural decision problem, "Decide if the list has a maximum" is trivial since every list has a maximum (there may be a tie), and so is $\mathcal{O}(1)$. But the natural search variant, "Find the maximum" requires looking through the list and so is $\mathcal{O}(n)$.
- V **V.4.13** True. Such problems have solution algorithms that are basically a fixed number of if ... then ... branches, so there is a maximum number of steps that any input can trigger. Thus, the solution algorithm is $\mathcal{O}(1)$.
- V **V.4.14** The collection P doesn't hold algorithms, it holds problems.
Sometimes people confuse 'runs in polytime' with 'is in P '. This is a type error because the type of thing that runs in polytime is different than the type of thing that is in P . Your coworker could say either "I've got a problem whose solution algorithm takes polytime" or "I've got a problem that is in P ."
- V **V.4.15** An order of growth is a set of functions. A complexity class is a set of language decision problems.
- V **V.4.16** Your friend is observing that the string representing a circuit may have length exponential in the circuit depth, since an r -deep circuit may have about 2^{r+1} many gates. But the definition of polynomial time is that the number of steps taken is a polynomial function of the size of the input representation, not of the number of circuit elements. A four-deep circuit takes four steps.
- V **V.4.17** The student's machine accepts all inputs. Restated, the student is proposing a machine that decides Σ^* . What's required by the definition is a machine that decides $\mathcal{L}$, that determines whether or not the input string is in the given language.
- V **V.4.18** True; in Table 1.27 the logarithmic functions are before the polynomials.
- V **V.4.19** True, although not by the same machine. Let $\mathcal{L}$ be computed by $\mathcal{P}$ so that the machine has this input-output behavior.

$$\phi_{\mathcal{P}}(\sigma) = \mathbb{1}_{\mathcal{L}}(\sigma) = \begin{cases} 1 & \text{-- if } \sigma \in \mathcal{L} \\ 0 & \text{-- otherwise} \end{cases}$$

Modify the machine by interchanging the output 0's and 1's, to get a new machine $\hat{\mathcal{P}}$ with the complementary behavior.

$$\phi_{\hat{\mathcal{P}}}(\sigma) = \mathbb{1}_{\mathcal{L}^c}(\sigma) = \begin{cases} 0 & \text{-- if } \sigma \in \mathcal{L} \\ 1 & \text{-- otherwise} \end{cases}$$

- V **V.4.20** A complexity class is just a set of languages. So the union of two is also a complexity class. The same holds for the intersection and for the complement, where the complement is taken inside the universe $\mathcal{P}(\mathbb{B}^*)$.
- V **V.4.21** A computably enumerable set is the range of some computable function ϕ_e . That matches the set of strings $\mathcal{L}_e = \{\text{str}(k) \mid k \in \mathbb{N} \text{ and } k = \phi_e(i) \text{ for some input } i \in \mathbb{N}\}$. The collection $RE = \{\mathcal{L}_e \mid e \in \mathbb{N}\}$ is a complexity class because it is a collection of languages.
- V **V.4.22** We must show that there is an algorithm to determine membership in this set that runs in polytime. Any straightforward algorithm will suffice. Here is one: given input σ , start by reversing it. Then compare the two strings σ and σ^R item by item. That algorithm runs in time $\mathcal{O}(|\sigma|)$.
- V **V.4.23** This is the decision problem for the language $\{\langle m, n \rangle \mid \text{the two are relatively prime}\}$. The obvious way to solve it is to use Euclid's algorithm to find whether their greatest common divisor is 1. That algorithm runs in polynomial time (in fact, in logarithmic time).
- V **V.4.24** In each case we produce an algorithm that decides membership in the language and that runs in time that is polynomial in the length of the input.

- (A) Given σ , decide if its length is divisible by three (determining the length takes about $|\sigma|$ steps). If not, return 0. If the length is divisible by three then set τ to be the substring that is the first third of σ (finding τ takes about $|\sigma|$ steps). Then just check, by moving through the string once, whether $\sigma = \tau^3$ (this also takes $|\sigma|$ steps). Return 1 if τ works and return 0 otherwise.

- (B) Stated as a language decision problem, we have $\{i \in \mathbb{N} \mid \mathcal{P}_i \text{ halts on the empty string within 10 steps}\}$. (We could instead explicitly use strings with $\{\text{str}(i) \mid i \in \mathbb{N} \text{ and } \mathcal{P}_i \text{ halts on the empty string in 10 steps}\}$.)

There is some overhead in going from the input index i to the Turing machine $\mathcal{P}_i$, in simulating it with a universal Turing machine, etc. But putting that aside as a constant overhead, no matter what is the input i , we then just run the simulation for ten steps. So the details of determining the exact exponent in the expression $\mathcal{O}(x^n)$ are ugly but this problem is an element of P .

V V.4.25

- (A) In the Clique problem, the size of the clique is a parameter. We are given a pair $\langle \mathcal{G}, B \rangle$ and asked if the graph has a B -sized clique. Here the size of the clique is fixed at $B = 3$.
- (B) Given $\mathcal{G}$, enumerate all triples $\langle v_0, v_1, v_2 \rangle \in \mathcal{N}^3$ and check whether all three of v_0v_1 , v_0v_2 , and v_1v_2 are edges, members of $\mathcal{E}$. This takes polynomial time. Specifically, where $|\mathcal{N}| = n$, there are $\binom{n}{2}$ many pairs v_iv_j ; that's about n^2 many. We check triples of pairs, so that's $\binom{n^2}{3}$, which is $\mathcal{O}(|\mathcal{N}|^6)$, which is polynomial. Lookups could add a factor of $|\mathcal{E}|$ but in any event it all happens in polytime.

V V.4.26

- (A) To determine whether the string is in alphabetical order, a single pass through it suffices. For each item except the initial one, we compare it to the prior one. Since the alphabet is given as $\Sigma = \{a, \dots, z\}$, comparisons are possible in polytime (for instance, a 26×26 lookup table is enough), and thus overall this one-pass algorithm takes polytime.
- (B) As mentioned in the discussion of the Primality problem, Problem 2.40, determining whether a number is composite or prime is known to be possible in polytime.
- (C) We can give an algorithm that runs in time polynomial in the length of the graph description. Fix the set $S_0 = \{v_0\}$. Next, follow each edge leading out of v_0 to get the set of vertices that are connected to v_0 in at most one step, $S_1 = S_0 \cup \{v \mid \{v_0, v\} \in \mathcal{E}\}$. After that, for all the vertices in S_1 follow all the edges to get the set $S_2 = S_1 \cup \{v \mid \{\hat{v}, v\} \in \mathcal{E} \text{ where } \hat{v} \in S_1\}$ of all vertices that are connected to v_0 in at most two steps. This process must stop at some point with $S_{i+1} = S_i$, because the graph is finite. Then, either $v_1 \in S_i$ or not, which determines whether there is a path from v_0 to v_1 . The number of steps in this algorithm, i , is bounded by the number of vertices in the graph and therefore the algorithm is polynomial in the size of the graph representation.

V V.4.27

- (A) By Dijkstra's algorithm, the Shortest Path problem can be done in polynomial time. So the problem is in P .
- (B) Knapsack is believed to be quite hard (in a later section we will see that it is NP complete). No one knows of a polynomial algorithm.
- (C) Euler Path is easy; we just need to check whether each node has even degree. There are algorithms that are linear.
- (D) No one knows of a polynomial algorithm for Hamiltonian Circuit. (In a later section we will see that it is NP complete.)

V.4.28 Yes. The algorithm that takes in a single input and returns 0 computes the characteristic function of the empty language, and clearly runs in polytime.

V V.4.29

- (A) Our definition of complexity class is completely general—it is a collection of languages. Under that definition, the collection of regular languages is a complexity class.
- (B) Yes. A language is regular if it is accepted by a Finite State machine. For every regular language we can make a Turing machine that simulates that Finite State machine and returns 0 or 1 to signal rejection or acceptance of the input strings. Each input string σ is processed by its Finite State machine in time $\mathcal{O}(|\sigma|)$ since the machine simply reads the character and passes to the indicated state. So the collection of regular languages is in P .

V V.4.30

Of course, P is the collection of languages that have a decider that runs in polytime. So the number

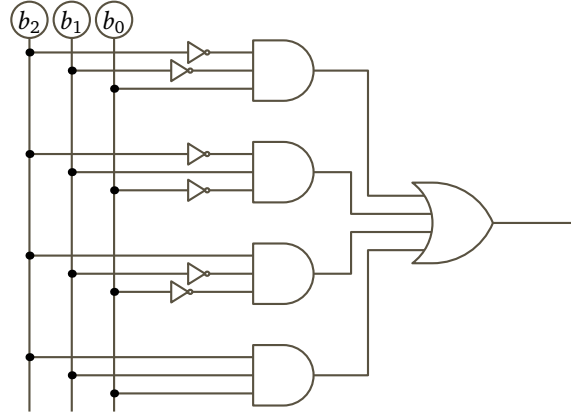


FIGURE 5, FOR QUESTION V.4.33: Initial circuit for the Circuit Evaluation problem.

of languages is bounded by the number of deciders. There are countably many Turing machines to use as deciders and thus P is countable.

- V **V.4.31** No. Take $\mathcal{L}_0 = \emptyset$ and $\mathcal{L}_1 = \mathcal{P}(\mathbb{B}^*)$. Because P is countable, as there are only countably many Turing machines to use as deciders, and because the collection of all languages $\mathcal{P}(\mathbb{B}^*)$ is uncountable, there are uncountably many languages with $\mathcal{L}_0 \subseteq \mathcal{L} \subseteq \mathcal{L}_1$. Consequently at least one such language is not in P .
- V **V.4.32** No. Certainly we can state it as a language decision problem, using the language $\mathcal{L} = \{n \in \mathbb{N} \mid \phi_n(n) \downarrow\}$ (to explicitly use strings, $\mathcal{L} = \{\text{str}(n) \mid n \in \mathbb{N} \text{ and } \phi_n(n) \downarrow\}$). But to be a member of P there must be a Turing machine that solves the problem (and that runs in polytime). No Turing machine decides Halting problem and so it is not in P .
- V **V.4.33** Here is the desired input-output behavior for the circuit.

b_2	b_1	b_0	$b_0 + b_1 + b_2$	$b_0 + b_1 + b_2 \pmod{2}$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	2	0
1	0	0	1	1
1	0	1	2	0
1	1	0	2	0
1	1	1	3	1

This propositional logic formula gives the behavior in that table (this is in Disjunctive Normal form).

$$(\neg b_2 \wedge \neg b_1 \wedge b_0) \vee (\neg b_2 \wedge b_1 \wedge \neg b_0) \vee (b_2 \wedge \neg b_1 \wedge \neg b_0) \vee (b_2 \wedge b_1 \wedge b_0)$$

Figure 5 on page 22 shows the circuit, using the standard notation of $\sqcap$ for ‘and’ gates, $\supset$ for ‘or’ gates, and $\triangleright$ for ‘not’ gates.

This circuit does not perfectly match the problem since some gates do not have two inputs and one output. We can parenthesize the propositional logic formula to make every operator 2-ary, taking advantage of the fact that $x_0 \wedge x_1 \wedge x_2 = (x_0 \wedge x_1) \wedge x_2$ and that a similar relationship holds for ‘ $\vee$ ’.

$$((\neg b_2 \wedge \neg b_1) \wedge b_0) \vee ((\neg b_2 \wedge b_1) \wedge \neg b_0) \vee ((b_2 \wedge \neg b_1) \wedge \neg b_0) \vee ((b_2 \wedge b_1) \wedge b_0)$$

Figure 6 on page 23 modifies the prior diagram to reflect that.

To give an acyclic directed graph we must take care of all four clauses. For the $(\neg b_2 \wedge \neg b_1) \wedge b_0$ first clause, the inside parenthesis needs a gate that does $\neg P \wedge \neg Q$, which we call the Boolean function f_0 . Then we combine $f_0(b_2, b_1)$ with b_0 using a $\wedge$. For the second clause $(\neg b_2 \wedge b_1) \wedge \neg b_0$, the inside parenthesis needs a gate that does $\neg P \wedge Q$, which we call the Boolean function f_1 . Then we combine $f_1(b_2, b_1)$ with b_0 using a

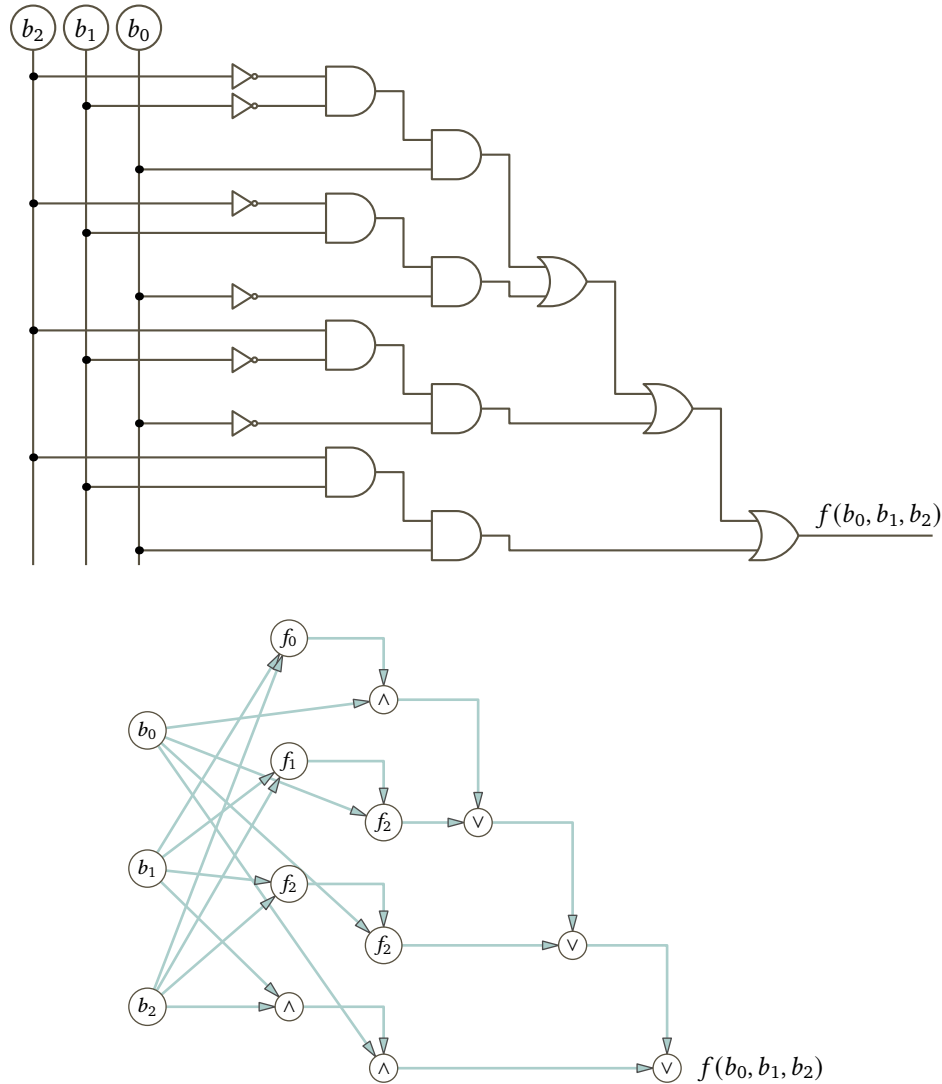


FIGURE 6, FOR QUESTION V.4.33: Revised circuit for the Circuit Evaluation problem and the associated DAG.

gate that implements the Boolean function $P \wedge \neg Q$, which we call f_2 . For the third clause $(b_2 \wedge \neg b_1) \wedge \neg b_0$, the inside parenthesis needs f_2 and then we combine $f_2(b_2, b_1)$ with b_0 using the f_2 function again. The fourth clause uses only $\wedge$'s. Here are the Boolean functions used there.

P	Q	$\wedge$	P	Q	$\vee$	P	Q	f_0	P	Q	f_1	P	Q	f_2
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	0	0	1	1	0	1	0	0	1	1	0	1	0
1	0	0	1	0	1	1	0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	1	0	1	1	0	1	1	0

V V.4.34

- (A) Suppose that $\mathcal{L}_0 \in P$ has the characteristic function ϕ_{e_0} that runs in time $\mathcal{O}(x^{k_0})$, and suppose that $\mathcal{L}_1$'s characteristic function ϕ_{e_1} runs in time $\mathcal{O}(x^{k_1})$. The canonical algorithm for intersection is to first test the input for membership in $\mathcal{L}_0$ and if it passes then go on to test for membership in $\mathcal{L}_1$. This algorithm is in $\mathcal{O}(x^k)$ where $k = \max(\{k_0, k_1\})$. So this algorithm is polytime.
- (B) We must prove that if a language $\mathcal{L}$ is in P then so is its set complement $\mathcal{L}^c$. If $\mathcal{L} \in P$ then there is a deterministic Turing machine $\mathcal{P}$ that runs in polynomial time, and that accepts an input string $\sigma \in \mathbb{B}^*$ if

and only if $\sigma \in \mathcal{L}$. (Recall that we take ‘accepts’ to mean that the machine halts, and when it halts there is only a 1 on the tape, while ‘rejects’ means there is only a 0.) Restated, there is a computable function $f: \mathbb{B}^* \rightarrow \mathbb{B}^*$ such that

$$f(\sigma) = \begin{cases} 0 & \text{if } \sigma \notin \mathcal{L} \\ 1 & \text{if } \sigma \in \mathcal{L} \end{cases}$$

and such that computing f is $\mathcal{O}(p)$ for some polynomial p . (More precisely, what is $\mathcal{O}(p)$ is the function $t(n)$, defined as the maximum runtime of $f(\tau)$ over all $\tau \in \mathbb{B}^*$ with $n = |\tau|$.)

Then clearly the following function is also computable, and also in polytime: $\bar{f}(\sigma) = 1 - f(\sigma)$. Consequently, $\mathcal{L}^c \in P$.

V V.4.35 Fix a language $\mathcal{L} \in P$. Then there is a Turing machine that computes the characteristic function $f: \mathbb{B}^* \rightarrow \mathbb{B}^*$ of $\mathcal{L}$, with runtime $\mathcal{O}(x^n)$ for some power $n \in \mathbb{N}$. To compute the characteristic function of the reversal, $\mathcal{L}^R = \{\tau^R \mid \tau \in \mathcal{L}\}$, the algorithm is given an input string τ . It constructs the reversal, τ^R , and decides whether $\sigma^R \in \mathcal{L}$. The algorithm can construct the reversal from the input in one pass through the input, and deciding whether that reversal string is in $\mathcal{L}$ is $\mathcal{O}(x^n)$, so in total the runtime is $\mathcal{O}(x^k)$ where $k = \max(\{1, n\})$.

V V.4.36 Fix two languages $\mathcal{L}_0, \mathcal{L}_1 \in \mathcal{P}(\mathbb{B}^*)$ that are members of P . Then there are exponents $n_0, n_1 \in \mathbb{N}$ so that $\mathcal{L}_0$ is accepted by a Turing machine with runtime $\mathcal{O}(x^{n_0})$, and $\mathcal{L}_1$ is accepted by a Turing machine with runtime $\mathcal{O}(x^{n_1})$.

Here is the natural algorithm to decide if a given input string $\sigma \in \mathbb{B}^*$ is a member of $\mathcal{L}_0 \cap \mathcal{L}_1$. We will pass through the string, first decomposing it as $\sigma = \varepsilon \sigma$, then as $\sigma = \sigma[0] \sigma[1:]$, etc. At each stage of this pass we have $\sigma = \alpha \beta$ and we check whether $\alpha \in \mathcal{L}_0$ and $\beta \in \mathcal{L}_1$.

Checking whether $\alpha \in \mathcal{L}_0$ takes time $\mathcal{O}(|\alpha|^{n_0})$, and checking whether $\beta \in \mathcal{L}_1$ takes time $\mathcal{O}(|\beta|^{n_1})$. (It simplifies things to overestimate and use $|\sigma|$, rather than worry about the length of the substrings.) Hence each step of the pass can be done in time that is $\mathcal{O}(|\sigma|^k)$ where $k = \max(\{n_0, n_1\})$. The pass itself takes $|\sigma| + 1$ many steps, so overall the algorithm runs in polytime, $\mathcal{O}(|\sigma|^m)$ where $m = 1 + \max(\{n_0, n_1\})$.

V V.4.37 This calls for an argument by induction.

The base step is that if two languages $\mathcal{L}_0, \mathcal{L}_1 \in \mathcal{P}(\mathbb{B}^*)$ are elements of P then their union is also an element of P . We established that in the proof of Lemma 4.12.

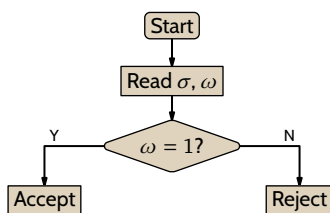
The inductive step takes as the hypothesis that for i -many languages with $2 < i < n$, their union $\mathcal{L}_0 \cup \dots \cup \mathcal{L}_i$ is in P . Consider $n + 1$ -many. The union is $\mathcal{L}_0 \cup \dots \cup \mathcal{L}_{i+1} = (\mathcal{L}_0 \cup \dots \cup \mathcal{L}_i) \cup \mathcal{L}_{i+1}$. This is the union of two members of P . By the first paragraph it is therefore a member of P .

As to a union of infinitely many, note that a single-element language $\mathcal{L} = \{n\}$ for $n \in \mathbb{N}$ is in P because a decider is basically a single if .. then .. statement and so runs in time $\mathcal{O}(1)$. Now any language is the union of infinitely many single-element languages. This includes any language that is not in P . Hence, P is not closed under the union of infinitely many languages.

V V.4.38 Lemma 4.12 says that P is closed under complementation. That is, if $\mathcal{L} \in P$ then $\mathcal{L}^c \in P$. It follows immediately that $P \subseteq coP$.

The inclusion $coP \subseteq P$ also follows from that same result, because if $L^c \in P$ then since $\mathcal{L}^{cc} = \mathcal{L}$, we have $\mathcal{L} \in P$.

V V.5.16 That lemma requires that the verifier have the property that if $\sigma \in \mathcal{L}$ then there is a witness leading to acceptance, while if $\sigma \notin \mathcal{L}$ then there does not exist a witness that would cause the verifier to accept. The proposed verifier doesn't verify, it relies on whatever ω is.



Because it does not go through the check, it will accept an input σ if the bit tells it to, even if σ is not in the language.

For a concrete instance, consider the problem $\mathcal{L} = \{n \in \mathbb{N} \mid n \text{ is even}\}$. The proposed verifier inputs a pair $\langle \sigma, \omega \rangle$, where σ is a number and ω is a bit, and then accepts the pair if $\omega = 1$. Absolutely, given the pair $\langle 5, 0 \rangle$, the verifier will not accept, which is desirable here because $5 \notin \mathcal{L}$. However, given $\langle 5, 1 \rangle$, the verifier will accept. So, for $\sigma = 5$ there exists an ω leading to acceptance, which violates the definition of a verifier.

V **V.5.17** It is ‘Every branch rejects’.

V **V.5.18** The easiest way to check satisfiability is to make a truth table.

(A) The expression $(P \wedge Q) \vee (\neg Q \wedge R)$ is satisfiable.

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

(B) The expression $(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$ is not satisfiable.

P	Q	$P \rightarrow Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg((P \wedge Q) \vee \neg P)$	$(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V **V.5.19** This machine does not decide any language. To be a decider, a nondeterministic Turing machine’s computation tree must have that on all inputs all maximal paths are finite.

V **V.5.20** No. The definition is: the machine accepts the input if there exists a maximal path that ends in an accepting configuration. Being a little bit informal or possibly evocative, we say that the machine accepts its input if there is a way for it to guess a path that triggers acceptance.

That definition means that the two models agree on their conclusions. The unboundedly parallel model is about all paths while the machine guesses model is about there exists a path, but we need them to agree on when a machine accepts its input and when it rejects.

V **V.5.21**

(A) Backtracking will solve the problem. But backtracking is a deterministic algorithm. (True, a deterministic algorithm is a special case of a nondeterministic one, but when you ask about it, your professor says, “The point of this exercise is to demonstrate understanding of nondeterminism, not to quibble.”)

(B) Your algorithm chooses, but it does so at random. If there are paths through the maze, but the random choices happens not to lead to such a path then your algorithm will not detect them. A nondeterministic algorithm is required to accept the input maze if there is a maximal path that ends in an accepting configuration and to not accept the input if there is not.

(C) We will give two answers, one using the mental model of unbounded parallelism and one using choosing.

For the unbounded parallelism model, whenever the algorithm finds itself at a place in the maze where it can do more than one thing, it forks child processes to handle each possibility. So it may have lots of branches, and lots of them may go to dead ends, but if there is a way through the maze then this algorithm will find it. It accepts the input maze if there is even a single path through and never accepts when there is not.

The algorithm for choosing will choose. That is, the machine guesses the path. (Instead of choosing, we can say that it receives the path ω from a demon.) By definition of the mental model, if there is a way for the machine to guess right then the machine accepts the input. (Note again that this algorithm is not guessing at random. It is guessing correctly.)

- V **V.5.22** True. By Lemma 5.8 it suffices to show that the language decision problem is in P . But that's clear: given a triple $\langle a, b, c \rangle$, checking whether the first two sum to the third is clearly a polytime task.
- V **V.5.23** In both models the input is an array $\langle a_0, a_1, \dots, a_{n-1} \rangle$.

In the unbounded parallelism model, the machine forks a child process for each index $0 \leq i < n$. Child i checks whether $a_i = k$, and if so signals the machine to accept.

The guessing model has the machine guess at an index i , and if it finds that $a_i = k$ there then it signals for acceptance. Under this model the machine accepts the same arrays as under the other because, by definition, the machine accepts an input if there is a maximal path that ends in an accepting configuration, that is, if at least one guess would be correct. If the array has a k entry then there is a way to guess it and the nondeterministic machine accepts that array. If the array has no such entry then there is a no way to guess an index leading to k and the nondeterministic machine rejects the input array.

On a deterministic machine the runtime is linear, $\mathcal{O}(n)$. In both nondeterministic models the runtime is constant, $\mathcal{O}(1)$. For instance in the unboundedly parallel model the machine looks in every entry but not in succession, instead all at the same time.

- V **V.5.24** To recast, we can denote each language decision problem as a tuple

$$\langle d_0, \dots, d_n, a_{0,0}, \dots, a_{0,n}, b_0, \dots, a_{m,0}, \dots, a_{m,n}, b_m \rangle \quad \text{each number is an integer}$$

representing that $F(x_0, \dots, x_n) = d_0x_0 + \dots + d_nx_n \geq B$ and that $a_{i,0}x_0 + \dots + a_{i,n}x_n \leq b_i$ for $0 \leq i \leq m$. (Observe that we can represent a ' $\geq$ ' constraint with negative numbers.)

An unboundedly parallel algorithm tries all feasible assignments of integers to the variable x_j 's. (An assignment is 'feasible' if it satisfies all of the constraints.) If there is a satisfying assignment then this algorithm will find it and accept the input tuple. If there is no satisfying assignment then there is no way for the input to get accepted.

A guessing algorithm guesses an assignment $\langle x_0, \dots, x_n \rangle$. If there is a satisfying assignment then there is a way to guess it, and so by the definition of machine acceptance under this model, the machine accepts the input tuple as a member of the language $\mathcal{L}_B$.

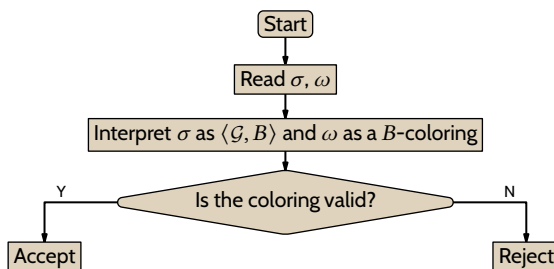
- V **V.5.25** The language is $\mathcal{L} = \{n \in \mathbb{N} \mid n \text{'s prime factorization is } n = p_0^{e_0} p_1^{e_1} \text{ for } e_0, e_1 > 0 \text{ and } p_0 \neq p_1 \text{ primes}\}$.

An unbounded parallelism algorithm tries each conceivable prime factorization, simultaneously (by 'conceivable' we mean that it doesn't include any primes greater than the input number). For instance, given the input 8, it could check the products $2^0 3^0 5^0 7^0$, $2^1 3^0 5^0 7^0$, $\dots$, $2^7 3^7 5^7 7^7$, to see whether any of them equals 8 and exactly two primes have nonzero exponents. Multiplying is fast and checking whether a number is composite or prime can also be done in polytime. Thus each of these checks happens in polytime. Since the machine runs them all at the same time, overall this runs in polytime.

A guessing algorithm is that the machine inputs the number n and guesses a two-prime factorization, or is given one by a demon. For instance, by the definition of this model the machine accepts the input 45, because it could guess $3^2 5^1$. If the input is not a semiprime then there is no way to guess a suitable factorization. Clearly this runs in polytime.

V.5.26

- (A) The language is a straightforward translation of the problem, $\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ is planar and is } B \text{ colorable} \}$.
 (B) Take the witness to be a coloring of the given graph with B -many colors.
 (C)



- (D) Lemma 5.11 needs a deterministic verifier. Its first input σ is a pair $\langle \mathcal{G}, B \rangle$ of a graph and a nonzero natural

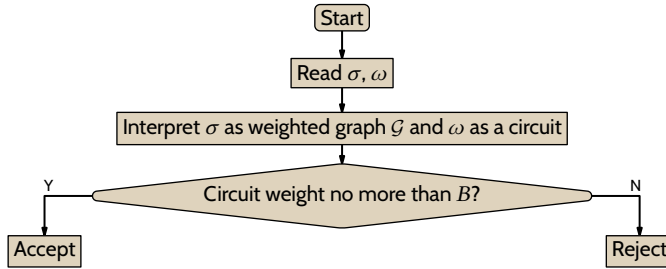
number. Its second input ω is the B -coloring of $\mathcal{G}$; we can think of this as the machine's guess, or the demon's hint.

If the graph has a B -coloring then there exists a suitable witness and the verifier $\mathcal{V}$ can check that using time polynomial in $|\sigma|$. If the graph does not have a B -coloring then then no ω will cause the verifier to accept its input.

- (E) Where $\mathcal{N}$ is the set of vertices of $\mathcal{G}$, there are at most $|\mathcal{N}|$ choose 2 many edges to check. That's $\mathcal{O}(|\mathcal{N}|^2)$, which is $\mathcal{O}(|\mathcal{G}|^2)$.

V.5.27

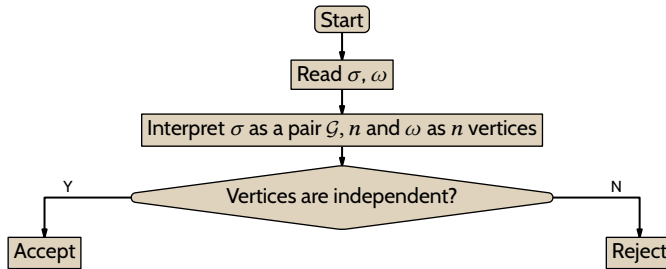
- (A) $\mathcal{L}_B = \{ \mathcal{G} \mid \mathcal{G} \text{ is a weighted graph with a circuit of length less than } B \}$
 (B) The witness is a circuit of weight no more than the bound B .
 (C)



- (D) Lemma 5.11 needs a deterministic Turing machine verifier $\mathcal{V}$. It must input pairs $\langle \sigma, \omega \rangle$. There, σ is a weighted graph $\mathcal{G}$ that is a candidate for membership in the language, and ω is a witness circuit of total cost no more than B . If $\sigma \in \mathcal{L}_B$ then there a circuit witness ω so that $\mathcal{V}$ accepts the input pair, while if $\sigma \notin \mathcal{L}_B$ then there is no ω that will verify and so $\mathcal{V}$ does not accept any input pair.
 (E) If the graph has n vertices then a circuit has $n - 1$ edges. Checking that the vertices inside the circuit are different takes n steps, and adding the edge weights is linear in the number of edges and the number of digits in the weights. So verification is polytime in $|\sigma|$.

V.5.28

- (A) $\mathcal{L} = \{ \langle \mathcal{G}, n \rangle \mid \text{the graph } \mathcal{G} \text{ has at least } n \text{ independent vertices} \}$
 (B) Given a graph and a natural number the witness is an independent set of graph vertices.
 (C)



- (D) Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. Its first input σ is a graph and natural number pair, $\langle \mathcal{G}, n \rangle$. Its second input ω is an n element independent set of $\mathcal{G}$'s vertices.

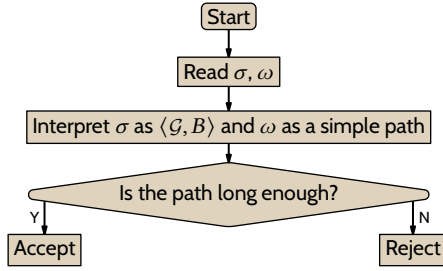
The verifier checks that the vertices are in the graph and that no two of them are connected. Those checks take time polynomial in $|\sigma|$.

If the pair $\langle \mathcal{G}, n \rangle$ is indeed an element of $\mathcal{L}$ then there is such a witness ω and so there exists a pair $\langle \sigma, \omega \rangle$ that the verifier accepts. If $\langle \mathcal{G}, n \rangle$ is not an element of $\mathcal{L}$ then there is no such ω and so there does not exist a pair that the verifier will accept.

- (E) The verifier must check ω 's vertices pairwise against the edges in the graph. The number of graph edges is polynomial in the number of vertices, which makes it polynomial in $|\sigma|$.

V.5.29

- (A) $\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ has a simple path of length at least } B \}$
 (B) For a specified graph and bound, the witness is a path of length no less than the bound.
 (C)



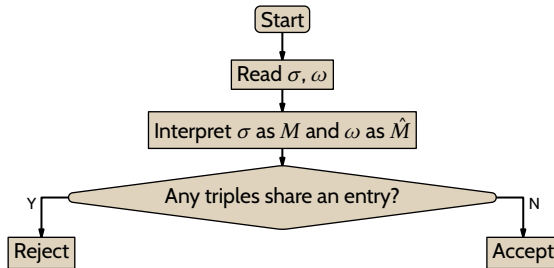
- (D) Lemma 5.11 wants a deterministic Turing machine verifier, $\mathcal{V}$. It inputs pairs $\langle \sigma, \omega \rangle$, where $\sigma = \langle \mathcal{G}, B \rangle$ and $\omega = \langle v_0, \dots, v_{n-1} \rangle$ is a simple path with no fewer than B edges.

The verifier checks that the path is in $\mathcal{G}$, that no two vertices are equal, and the the length at least B . These checks can be done in time polynomial in $|\sigma|$. If σ is indeed an element of $\mathcal{L}$ then there is such a path ω , so there exists a pair $\langle \sigma, \omega \rangle$ that the verifier accepts. If σ is not an element of $\mathcal{L}$ then there is no such ω and so there does not exist a pair that the verifier accepts.

- (E) Where the path ω has length $n - 1$, checking that the vertices are unequal can be done by traversing the path, so that has a runtime of $\mathcal{O}(n)$, which is clearly polynomial in $|\sigma|$.

V.5.30

- (A) The language is the collection of sets M that contain triples $M \subseteq X \times Y \times Z$ where $|X| = |Y| = |Z| = n$ for some $n \in \mathbb{N}^+$, and M has a subset $\hat{M}$ of size n such that no two of its triples agree on any if their three coordinates.
- (B) Given $\sigma = M$, take $\omega = \hat{M}$ if one exists.
- (C)



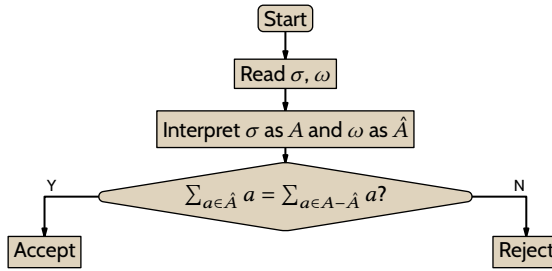
- (D) The verifier input σ is a set of triples M whose first coordinates are limited to some size n set X , whose second coordinates are limited to a size n set Y , and whose third coordinates are limited to a size n set Z . It also inputs the witness ω that is $\hat{M} \subseteq M$ satisfying the condition (we can think of this as the nondeterministic machine's guess or as the demon's hint).

If $\sigma = M$ has such a sub-multiset $\hat{M}$ then verifying that ω works can be done in time polynomial in $|\sigma|$. So in this case there is an input pair beginning with σ that the verifier accepts. If there is no such $\hat{M}$ for the input $\sigma = M$ then for no ω will the verifier find that it checks out, so it will not accept any input pairs.

- (E) Checking that no two triples have the same first coordinate requires $|\hat{M}|$ choose 2 comparisons, which is $\mathcal{O}(|\hat{M}|^2)$. To check all three coordinates takes three times as many steps, again $\mathcal{O}(|\hat{M}|^2)$. Because $\hat{M} \subseteq M$, that's $\mathcal{O}(|\sigma|^2)$.

V.5.31

- (A) $\mathcal{L} = \{A \mid A \text{ is a multiset of natural numbers with a sub-multiset } \hat{A} \subseteq A \text{ such that } \sum_{a \in \hat{A}} a = \sum_{a \in A - \hat{A}} a\}$
- (B) The witness ω is the sub-multiset $\hat{A}$.
- (C)



It is on the right of ?? on page ??.

- (D) Lemma 5.11 needs a deterministic verifier. It inputs σ , the problem instance M , and ω , the witness $\hat{A}$ (we can think of this as the nondeterministic machine's guess or as the demon's hint). If $\sigma \in \mathcal{L}$ then by definition such an $\hat{A}$ exists and so the verifier accepts at least one input pair beginning with σ (verification takes time polynomial in $|\sigma|$). If $\sigma \notin \mathcal{L}$ then there is no ω that will verify so $\mathcal{V}$ accepts no input pair beginning with σ .
- (E) Adding the two multisets of numbers takes time polynomial in their digits, which is polynomial in $|\sigma|$.

V.5.32

- (A) This is the language $\mathcal{L}$.

$$\{\sigma \mid \sigma \text{ represents a Propositional Logic statement where at least two lines of the truth tables end in } T\}$$

These are suitable fill-ins: (1) a Propositional Logic statement E , (2) a pair, pointing to two lines of the truth table whose final entry is T , and (3) evaluation of a fixed Propositional Logic statement takes time polynomial in the length of the statement, and thus polytime in $|\sigma|$.

- (B) Here is the problem cast as the decision problem for a language.

$$\mathcal{L} = \{\langle S, T \rangle \mid \text{a sub-multiset of } A \subseteq S \text{ has } \sum_{a \in A} a = T\}$$

These are fill-ins: (1) a pair $\langle S, T \rangle$ of a multiset and a number, (2) a sub-multiset $\hat{S}$ of S whose elements add to T , and (3) addition takes time polynomial in the number of inputs and $\hat{S} \subseteq S$ so that is time polynomial in $|\sigma|$.

- V.5.33** Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. It inputs pairs $\langle \sigma, \omega \rangle$, where σ is a problem instance $\langle s_0, \dots, s_5, T \rangle \in S^6 \times \{100, \dots, 999\}!$. The witness ω is an arithmetic expression that evaluates to the target number T and that involves a sub-multiset of the numbers $s_0, \dots, s_5$.

The verifier checks that the expression does indeed evaluate to the target, and does indeed use each of the six numbers at most once. Those checks can be done in time polynomial in $|\sigma|$. (Briefly: addition of two n -digit numbers is $\mathcal{O}(n)$ and so addition of six n -digit numbers is also linear.) If $\sigma \in \mathcal{L}$ then by definition there exists such an arithmetic expression and so $\mathcal{V}$ can accept an input pair. If $\sigma \notin \mathcal{L}$ then there is no such arithmetic expression, and therefore no witness ω will cause $\mathcal{V}$ to accept.

- V.5.34** The Knapsack problem is the decision problem for this language $\mathcal{L}$.

$$\{\langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle \mid \text{there is } I = \{i_0, \dots, i_k\} \subseteq \{0, \dots, n-1\} \text{ so } \sum_{i \in I} w_i \leq B \text{ and } \sum_{i \in I} v_i \geq T\}$$

The summary is that we take the witness to be a set of indices $\{i_0, \dots, i_k\} \subset \{0, \dots, n-1\}$ naming items that together meet the constraints.

For more detail: Lemma 5.11 asks for a deterministic Turing machine verifier, $\mathcal{V}$. Its first input σ is a problem instance $\langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle$. As described in the prior paragraph, its second input names items that meet the constraints B and T .

If $\sigma \in \mathcal{L}$ then there is a witness to that, ω , and $\mathcal{V}$ can verify it in time polynomial in $|\sigma|$. (For that argument: addition of two m digit integers is linear in the number of digits, $\mathcal{O}(m)$. When there are more potential items to add to the knapsack, the representation σ gets bigger, and verification remains linear in $|\sigma|$.) Consequently, in this case the verifier accepts its input. If $\sigma \notin \mathcal{L}$ then there is no such ω and the verifier will not accept any pair that begins with σ .

- V.5.35** For both we will use Lemma 5.11.

(A) Here is an appropriate language.

$$\mathcal{L} = \{ \langle a, b \rangle \in \mathbb{N}^2 \mid \text{there exists } x \in \mathbb{N} \text{ with } ax + 1 = b \}$$

The short version of the argument is that we can take the witness to be a suitable x .

For more, Lemma 5.11 asks for a deterministic Turing machine verifier, $\mathcal{V}$. Its first input σ is the number pair $\langle a, b \rangle$. Its second input ω is a number x so that $ax + 1 = b$ if there is one.

In the case that a and b are such that there is such an x then The verifier can check that ω works, that is, that that $ax + 1 = b$. Because arithmetic can be done in polytime, the verifier can do this in time polynomial in $|\sigma|$. In the other case that for the give a and b there is no such number then there is no input pair beginning with σ that causes the verifier to accept its input.

(B) This is the associated language.

$$\mathcal{L} = \{ M \mid \text{there is a set } P \subset \mathbb{Z} \text{ of numbers such that } M \text{ is the multiset of pairwise distances} \}$$

The summary: given a multiset problem instance $\sigma = M$, take the witness $\omega = P$.

More detail is that we want a deterministic Turing machine verifier $\mathcal{V}$ that inputs pairs $\langle \sigma, \omega \rangle$, where σ is a multiset M . As a witness we take a set of positions, $\omega = P$.

If $\sigma \in \mathcal{L}$ then there is such a set of positions P , so there exists a pair $\langle \sigma, \omega \rangle$ that the verifier accepts. (The check that the pairwise distances among elements of P equals M . can be done in time polynomial in $|\sigma|$ because arithmetic can be done in polytime.) On the other hand, if $\sigma \notin \mathcal{L}$ then there is no such ω and so there does not exist an input pair that the verifier accepts.

V V.5.36 We use Lemma 5.11. This is an appropriate language $\mathcal{L}_B$, where the road map is the weighted graph $\mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle$.

$$\{ V \subseteq \mathcal{N} \mid \text{there are two cycles, whose weights are each at most } B \text{ and whose vertices together make } V \}$$

A summary of the proof is: given V , a suitable witness is a pair $\langle C_0, C_1 \rangle$ of two cycles.

To use Lemma 5.11 we must produce a verifier. It inputs $\langle \sigma, \omega \rangle$, where $\sigma = V$. If $\sigma \in \mathcal{L}$ then there is a suitable pair of cycles, so there exists a witness ω such that the verifier accepts the input pair. (Verification of the conditions clearly can be done in time polynomial in $|\sigma|$, because arithmetic can be done in polytime.) If $\sigma \notin \mathcal{L}$ then there are not two such cycles, so there is no suitable ω , and so the verifier will not accept any input pair starting with σ .

V V.5.37 The proof is much like Theorem 5.4's. Here also one direction is easy because a deterministic Turing machine is a special case of a nondeterministic one, so if a deterministic machine recognizes a language then a nondeterministic one does also.

For the converse suppose that a nondeterministic Turing machine $\mathcal{P}$ recognizes a language $\mathcal{L}$. We will describe a deterministic machine $\mathcal{Q}$ that does the same.

Fix an input σ . Have the deterministic machine $\mathcal{Q}$ do a breadth-first search of $\mathcal{P}$'s computation tree. If $\mathcal{P}$ accepts σ then there is a maximal path whose terminal configuration has an accepting state, and $\mathcal{Q}$ will eventually find that configuration because the search is breadth-first. If the tree has no accepting maximal path then $\mathcal{Q}$ will never find one and so will not accept σ .

V V.5.38

(A) The most natural language is $\mathcal{L} = \{ \langle \mathcal{G}_0, \mathcal{G}_1 \rangle \mid \text{they are isomorphic} \}$.

(B) We will show that the problem is in *NP* using Lemma 5.11. The summary is: given the problem instance $\sigma = \langle \mathcal{G}_0, \mathcal{G}_1 \rangle$, choose as witness the function $\omega = f$ that is the isomorphism.

In more detail, we must produce a deterministic Turing machine verifier. It inputs $\langle \sigma, \omega \rangle$ where σ is a pair of graphs, $\langle \mathcal{G}_0, \mathcal{G}_1 \rangle$. If $\sigma \in \mathcal{L}$ then there is an isomorphism map and so there is a witness ω so that the verifier can accept its input pair. (Checking that the function is one-to-one and onto, and that $\{v, \hat{v}\}$ is an edge of $\mathcal{G}_0$ if and only if $\{f(v), f(\hat{v})\}$ is an edge of $\mathcal{G}_1$ can clearly be done in time polynomial in $|\sigma|$.) If $\sigma \notin \mathcal{L}$ then there is no such function, so no witness ω will check out and the verifier never accepts an input pair that begins with σ .

V V.5.39 Countable. A verifier is a deterministic Turing machine, and there are countably many such machines.

V V.5.40

(A) The class NP is the set of languages decidable by a nondeterministic machine in polytime. Every problem decidable by a nondeterministic Turing machine is decidable by a deterministic Turing machine. The Halting problem is not decidable by a deterministic Turing machine, so it is not in NP .

(B) There is no polynomial bound on the number of steps, and consequently on the length of ω .

- V **V.5.41** A configuration of a nondeterministic Turing machine $\mathcal{P}$ is a four-tuple, $\langle q, s, \tau_L, \tau_R \rangle \in Q \times \Sigma \times \Sigma^* \times \Sigma^*$. The items signify the current state q , the character under the read/write head s , and the tape contents to the left and right of the head τ_L and τ_R . Where the machine's input is the string σ , if that string's first character is s , so that $\sigma = \langle s \rangle \frown \alpha$, then the computation's initial configuration is $\mathcal{C}_0 = \langle q_0, s, \varepsilon, \alpha \rangle$. (If the input is empty then we take $s = B$.)

We next describe under what circumstances a configuration $\hat{\mathcal{C}}$ succeeds $\mathcal{C}$, denoted $\mathcal{C} \vdash \hat{\mathcal{C}}$. Suppose that $\mathcal{C} = \langle q, s, \tau_L, \tau_R \rangle$. Compute the set $\Delta(q, s)$. If that set is empty then there is no next configuration $\hat{\mathcal{C}}$.

Otherwise that set is not empty. For every instruction $\langle q_p T_p T_n q_n \rangle \in \Delta(q, s)$ there are three possibilities. Exactly one of those three holds and it defines a succeeding configuration $\hat{\mathcal{C}}$. (1) If the action symbol T_n is a member of the tape alphabet, $T_n \in \Sigma$, then write it to the tape, $\hat{\mathcal{C}} = \langle q_n, T_n, \tau_L, \tau_R \rangle$. (2) If T_n is the character L then move the tape head to the left. That is, $\hat{\mathcal{C}} = \langle q_n, \hat{s}, \hat{\tau}_L, \hat{\tau}_R \rangle$ where $\hat{s}$ is the rightmost character of the string τ_L (if $\tau_L = \varepsilon$ then $\hat{s}$ is the blank character, B), where $\hat{\tau}_L$ is τ_L with its rightmost character omitted (if $\tau_L = \varepsilon$ then $\hat{\tau}_L = \varepsilon$ also), and where $\hat{\tau}_R$ is the string formed by pushing the tape character s onto the front of τ_R , giving $\hat{\tau}_R = \langle s \rangle \frown \tau_R$. (3) If $T_n = R$ then move the tape head to the right. This is so much like the prior item that we omit the details.

A computation path is a sequence of configurations $\mathcal{C}_0 \vdash \mathcal{C}_1 \vdash \dots$, starting with the initial configuration. The union of all such paths forms a directed graph called the computation tree, or simply the computation. If for a given input σ at least one maximal path in that tree ends in a configuration whose state is accepting then we say that the machine accepts σ . If the machine does not accept σ then it rejects σ .

- V **V.6.10** The first way is right: when $\mathcal{L}_1 \leq_p \mathcal{L}_0$ then $\mathcal{L}_1$ reduces to $\mathcal{L}_0$.
- V **V.6.11** No. That intuition fits Cook reduction. Karp reduction make a call to the black box for A at most one time. This is the ' $f(\sigma) \in \mathcal{L}_0$ ' at the end of Definition 6.1.
- V **V.6.12** It is the first. A fast algorithm for $\mathcal{L}_0$ would give a fast one for $\mathcal{L}_1$.
- V **V.6.13** The assertion about P is immediate from Lemma 6.9's item (4). It also holds for NP by item (5).
- V **V.6.14** If $\mathcal{L}_1 \leq_p \mathcal{L}_0$ via the reduction function f then there can be another language $\mathcal{L}$ where $\mathcal{L}_1 \leq_p \mathcal{L}$ via the same reduction function f . There can be uncountably many such other languages.

Expanding a bit: in the proof of Lemma 6.9's argument for item (3), the reduction function is as below for the input string α , using one member $\sigma \in \mathcal{L}_0$ and one nonmember $\tau \notin \mathcal{L}_0$.

$$f(\alpha) = \begin{cases} \sigma & \text{if } \alpha \in \mathcal{L}_1 \\ \tau & \text{otherwise} \end{cases}$$

This definition leaves untouched the question of elementhood in $\mathcal{L}$ for all strings that are not equal to σ or τ . So this one reduction function relates $\mathcal{L}_1$ to uncountably many other sets.

- V **V.6.15** There is indeed a subset that adds to T , namely $A = \{23, 31, 72\}$.
- V **V.6.16** Lemma 6.9 says that $\leq_p$ is a reflexive relation so the diagonal below contains Y's. By the same lemma, any problem in P is polytime reducible to any other problem at all. So the first, second, and fourth row are all Y's.

	$\mathcal{L}_0$	$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$
$\mathcal{L}_0$	Y	Y	Y	Y
$\mathcal{L}_1$	Y	Y	Y	Y
$\mathcal{L}_2$	N	N	Y	N
$\mathcal{L}_3$	Y	Y	Y	Y

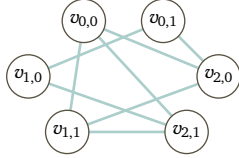
Also by the lemma, if a problem is polytime reducible to one in P then it is itself in P . But $\mathcal{L}_2$ is given as not in P , which accounts for the N's in the third row.

- V **V.6.17** For reference the expression is $(x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1) \wedge (x_0 \vee \neg x_1)$.

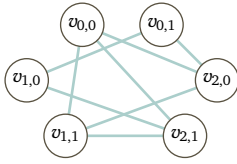
Example 6.5 says that we want the graph $\mathcal{G}$ whose vertices are pairs $\langle c, \ell \rangle$ where c is the number of a clause and ℓ is a literal in that clause. Two vertices $v_0 = \langle c_0, \ell_0 \rangle$ and $v_1 = \langle c_1, \ell_1 \rangle$ are connected by an edge if they come from different clauses, $c_0 \neq c_1$, and the literals are not negations of each other, $\ell_0 \neq \neg \ell_1$.

The given propositional logic expression has three clauses, $x_0 \vee x_1$, and $\neg x_0 \vee \neg x_1$, and $x_0 \vee \neg x_1$, each with two literals. This is the graph.

$$\begin{aligned} v_{0,0} &= \langle 0, x_0 \rangle \\ v_{0,1} &= \langle 0, x_1 \rangle \\ v_{1,0} &= \langle 1, \neg x_0 \rangle \\ v_{1,1} &= \langle 1, \neg x_1 \rangle \\ v_{2,0} &= \langle 2, x_0 \rangle \\ v_{2,1} &= \langle 2, \neg x_1 \rangle \end{aligned}$$



V V.6.18 For reference here is the graph.



For each vertex there is a clause.

$$a_{0,0} \vee b_{0,0} \vee c_{0,0} \quad a_{0,1} \vee b_{0,1} \vee c_{0,1} \quad a_{1,0} \vee b_{1,0} \vee c_{1,0} \quad a_{1,1} \vee b_{1,1} \vee c_{1,1} \quad a_{2,0} \vee b_{2,0} \vee c_{2,0} \quad a_{2,1} \vee b_{2,1} \vee c_{2,1}$$

For each edge there are three clauses. For instance, for the edge between $v_{0,0}$ and $v_{1,1}$ the three clauses are here.

$$(\neg a_{0,0} \vee \neg a_{1,1}) \quad (\neg b_{0,0} \vee \neg b_{1,1}) \quad (\neg c_{0,0} \vee \neg c_{1,1})$$

Construct the entire expression,

$$\begin{aligned} E = & (a_{0,0} \vee b_{0,0} \vee c_{0,0}) \wedge (a_{0,1} \vee b_{0,1} \vee c_{0,1}) \wedge (a_{1,0} \vee b_{1,0} \vee c_{1,0}) \\ & \wedge (a_{1,1} \vee b_{1,1} \vee c_{1,1}) \wedge (a_{2,0} \vee b_{2,0} \vee c_{2,0}) \wedge (a_{2,1} \vee b_{2,1} \vee c_{2,1}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{1,1}) \wedge (\neg b_{0,0} \vee \neg b_{1,1}) \wedge (\neg c_{0,0} \vee \neg c_{1,1}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{2,0}) \wedge (\neg b_{0,0} \vee \neg b_{2,0}) \wedge (\neg c_{0,0} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{2,1}) \wedge (\neg b_{0,0} \vee \neg b_{2,1}) \wedge (\neg c_{0,0} \vee \neg c_{2,1}) \\ & \wedge (\neg a_{0,1} \vee \neg a_{1,0}) \wedge (\neg b_{0,1} \vee \neg b_{1,0}) \wedge (\neg c_{0,1} \vee \neg c_{1,0}) \\ & \wedge (\neg a_{0,1} \vee \neg a_{2,0}) \wedge (\neg b_{0,1} \vee \neg b_{2,0}) \wedge (\neg c_{0,1} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{1,0} \vee \neg a_{2,1}) \wedge (\neg b_{1,0} \vee \neg b_{2,1}) \wedge (\neg c_{1,0} \vee \neg c_{2,1}) \\ & \wedge (\neg a_{1,1} \vee \neg a_{2,0}) \wedge (\neg b_{1,1} \vee \neg b_{2,0}) \wedge (\neg c_{1,1} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{1,1} \vee \neg a_{2,1}) \wedge (\neg b_{1,1} \vee \neg b_{2,1}) \wedge (\neg c_{1,1} \vee \neg c_{2,1}) \end{aligned}$$

by taking the conjunction of the clauses.

V V.6.19

- (A) Three are: $\beta_0 = \alpha = 0110010$, $\beta_1 = 1100100$, and $\beta_2 = 1001001$.
 (B) It is a cyclic shift starting from index $k = 6$.
 (C) It is the decision problem for this language.

$$\mathcal{L}_0 = \{ \langle \sigma, \tau \rangle \in \Sigma^2 \mid \tau \text{ is a substring of } \sigma \}$$

- (D) It is the decision problem for this.

$$\mathcal{L}_1 = \{ \langle \alpha, \beta \rangle \in \Sigma^2 \mid \beta \text{ is a cyclic shift of } \alpha \}$$

- (E) Following the hint, the reduction function f inputs $\langle \alpha, \beta \rangle$ and if the two are the same length then it outputs $\langle \alpha \frown \alpha, \beta \rangle$. (If they are not the same length then it outputs some pair sure to fail, such as $\langle \varepsilon, \emptyset \rangle$.) Clearly $\langle \alpha, \beta \rangle \in \mathcal{L}_1$ if and only if $f(\langle \alpha, \beta \rangle) \in \mathcal{L}_0$. Also clear is that f is computable in polytime. Therefore $\mathcal{L}_1 \leq_p \mathcal{L}_0$.

V V.6.20

- (A) The Hamiltonian Circuit problem is the decision problem for this language.

$$\mathcal{L}_0 = \{ \mathcal{H} \mid \text{this unweighted graph contains a circuit that includes each vertex exactly once} \}$$

The Traveling Salesman problem is this decision problem.

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, B \rangle \mid \text{some circuit of the weighted graph } \mathcal{G} \text{ that includes each vertex has total cost at most } B \}$$

- (B) The reduction function inputs an instance of Hamiltonian Circuit, a graph $\mathcal{H} = \langle \mathcal{N}, \mathcal{E} \rangle$ whose edges are unweighted. It returns an instance of Traveling Salesman, a pair $\langle \mathcal{G}, B \rangle$. The first entry $\mathcal{G}$ is a weighted graph that uses $\mathcal{H}$'s vertex set $\mathcal{N}$ as cities and that takes the distances between the cities to be: $d(v_i, v_j) = 1$ if $v_i v_j$ is an edge of $\mathcal{H}$ and $d(v_i, v_j) = 2$ if $v_i v_j \notin \mathcal{E}$. The second entry in that pair is the bound $B = |\mathcal{N}|$.

Because of this bound, there will be a Traveling Salesman solution if and only if there is a Hamiltonian Circuit solution, namely where the Traveling Salesman solution uses the edges of the Hamiltonian circuit. That is, $\mathcal{N} \in \mathcal{L}_0$ if and only if $f(\mathcal{H}) = \langle \mathcal{G}, B \rangle \in \mathcal{L}_1$.

- (C) The number of edges is less than twice the number of vertices, so that polytime in the input graph size is the same as polytime in the number of vertices. To output the Traveling Salesman instance, the algorithm examines all the pairs of vertices, which is a nested loop and so takes time that is quadratic in the number of vertices. Quadratic in a polynomial is still polynomial. So f runs in polytime.

V V.6.21

- (A) This is the language for the 3-SAT problem.

$$\{ E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals} \}$$

This is the language for Strict 3-Satisfiability.

$$\{ E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has exactly three literals} \}$$

- (B) Any instance of Strict 3-Satisfiability is an instance of 3-SAT. So the reduction function is the identity function.
- (C) We are asked to show that the two-literal Propositional Logic expression $E_0 = P \vee Q$ is equivalent to the expression $E_1 = (P \vee Q \vee R) \wedge (P \vee Q \vee \neg R)$ where each clause has three literals.

The most straightforward thing is to use a truth table to compare the two expressions.

P	Q	R	E_0	$P \vee Q \vee R$	$P \vee Q \vee \neg R$	E_1
F	F	F	F	F	T	F
F	F	T	F	T	F	F
F	T	F	T	T	T	T
F	T	T	T	T	T	T
T	F	F	T	T	T	T
T	F	T	T	T	T	T
T	T	F	T	T	T	T
T	T	T	T	T	T	T

The same technique works for the one-literal expression $E_2 = P$. It is equivalent to the expression $E_3 = (P \vee Q \vee R) \wedge (P \vee \neg Q \vee R) \wedge (P \vee Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R)$ where each clause has three literals.

P	Q	R	E_2	$(P \vee Q \vee R)$	$(P \vee \neg Q \vee R)$	$(P \vee Q \vee \neg R)$	$(P \vee \neg Q \vee \neg R)$	E_3
F	F	F	F	F	T	T	T	F
F	F	T	F	T	T	F	T	F
F	T	F	F	T	F	T	T	F
F	T	T	F	T	T	T	F	F
T	F	F	T	T	T	T	T	T
T	F	T	T	T	T	T	T	T
T	T	F	T	T	T	T	T	T
T	T	T	T	T	T	T	T	T

- (D) The reduction function inputs expressions in Conjunctive Normal form $E = \sigma_0 \wedge \sigma_1 \wedge \dots \sigma_k$, where the σ_i clauses are disjunctions of three or fewer literals. It must output equivalent propositional logic expressions that also are in Conjunctive Normal form but where the clauses are disjunctions of exactly three literals. The prior item shows how the function converts the one to the other. Obviously the function can run in polytime.

V V.6.22 This is the language for the 3-SAT problem.

$$\{E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals}\}$$

This is the language for SAT.

$$\{E \mid E \text{ is a satisfiable Boolean expression in CNF}\}$$

- (A) The reduction function f must input an instance of 3-SAT, a propositional logic expression in Conjunctive Normal form where the clauses have at most three literals. It must output an instance of SAT, also a propositional logic expression in Conjunctive Normal form. So take f to be the identity $f(\sigma) = \sigma$. Obviously the input is satisfiable if and only if the output is also satisfiable. Obviously also, it runs in polytime.
- (B) First consider an assignment of the truth values T or F to the variables $P_0, \dots, P_4$ that makes $C = p_0 \vee p_1 \vee p_2 \vee p_3 \vee p_4$ evaluate to T . Then at least one of the literals $p_0, \dots, p_4$ evaluates to T . If either p_0 or p_1 is T then setting A to F makes $E = (A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$ evaluate to T . Otherwise, if C is T but neither p_0 nor p_1 is T , then setting A to T makes E evaluate to T .

Now suppose that no assignment of the truth values to the variables $P_0, \dots, P_4$ makes C evaluate to T . Then no matter what truth value we assign to A , one of E 's two clauses $(A \vee p_0 \vee p_1)$ and $(\neg A \vee p_2 \vee p_3 \vee p_4)$ is F . Hence for every assignment of truth values to $P_0, \dots, P_4$ and A the expression E gives F .

- (C) The reduction function inputs expressions S in Conjunctive Normal form. It must output Conjunctive Normal form expressions where the clauses have three or fewer literals.

The prior item suggests how the reduction function does that. It replaces each clause C having more than three literals with a number of clauses having three or fewer. For instance starting with input $p_0 \vee p_1 \vee p_2 \vee p_3 \vee p_4$, the reduction function can go to $(A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$, and then to the output $(A \vee p_0 \vee p_1) \wedge (B \vee \neg A \vee p_2) \wedge (\neg B \vee p_3 \vee p_4)$. Clearly that transformation can be done in polytime.

V V.6.23

- (A) The Independent Set problem is the decision problem for this language ($\mathcal{N}_G$ is the set of vertices of G).

$$\mathcal{L}_0 = \{\langle G, B \rangle \mid \text{there is } I \subseteq \mathcal{N}_G \text{ such that } |I| \geq B \text{ and no edge contains two members of } I\}$$

Vertex Cover is the decision problem for this language.

$$\mathcal{L}_1 = \{\langle \mathcal{H}, D \rangle \mid \text{there is } C \subseteq \mathcal{N}_H \text{ such that } |C| \leq D \text{ and every edge contains at least one member of } C\}$$

- (B) A vertex cover with $k = 4$ elements is $S = \{q_2, q_5, q_8, q_9\}$.
- (C) An independent set with $\hat{k} = 6$ elements is $\hat{S} = \{q_0, q_1, q_3, q_4, q_6, q_7\}$.
- (D) By definition, a set I of vertices is independent if every edge contains at most one element of I . Also by definition, a set C of vertices is a vertex cover if every edge contains at least one element of C .

Let I be an independent set. For all edges e , at most one end of e is in I . That implies that at least one end of e is in $\mathcal{N} - I$. So if I is an independent set then $\mathcal{N} - I$ is a vertex cover.

Similarly, let I be such that $\mathcal{N} - I$ is a vertex cover. Then for all edges $\hat{e}$, at least one end of $\hat{e}$ is a member of $\mathcal{N} - I$. Thus at most one end of $\hat{e}$ is a member of I . So if $\mathcal{N} - I$ is a vertex cover then I is an independent set.

Therefore a set of vertices is independent if and only if its complement is a vertex cover.

- (E) We will do **Independent Set** $\leq_p$ **Vertex Cover**; the other reduction is similar. By the prior item, there is a relationship between independent sets and vertex covers. In particular, if there is an independent set of size $\hat{k}$ then there is a vertex cover of size $|\mathcal{N}| - \hat{k}$.

The definition requires that we produce a polytime computable function f . Given an problem instance of **Independent Set** $\sigma = \langle \mathcal{G}, \hat{k} \rangle$, the function converts it to a problem instance of **Vertex Cover** by adjusting the bound, $f(\sigma) = \langle \mathcal{G}, |\mathcal{G}| - \hat{k} \rangle$. By the prior item, $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$. Also, clearly f can run in polytime.

V.6.24

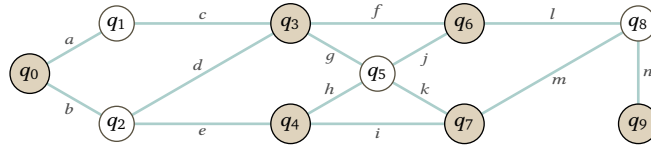
- (A) **Vertex Cover** is the decision problem for this language.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \text{there is } C \subseteq \mathcal{N}(\mathcal{G}) \text{ with } |C| \leq B \text{ such that every edge contains a vertex from } C \}$$

Set Cover is the decision problem for this language.

$$\mathcal{L}_1 = \{ \langle S, S_0, \dots, S_{k-1}, D \rangle \mid S_j \subseteq S \text{ for each } j, \text{ and there is } I \subset \{0, \dots, k-1\} \text{ with } |I| \leq D, \text{ and } \cup_{i \in I} S_i = S \}$$

- (B) One vertex cover with six elements is $C = \{q_0, q_3, q_4, q_6, q_7, q_9\}$.



- (C) The set of edges is $S = \{a, b, \dots, n\}$. The subsets are these.

$$S_0 = \{a, b\}, S_1 = \{a, c\}, S_2 = \{b, d, e\}, S_3 = \{c, d, f, g\}, S_4 = \{e, h, i\}, \\ S_5 = \{g, h, j, k\}, S_6 = \{f, j, l\}, S_7 = \{i, k, m\}, S_8 = \{l, m, n\}, S_9 = \{n\}$$

To find a set cover we can take the subsets associated with vertices in the vertex cover.

$$S_{q_0} \cup S_{q_3} \cup S_{q_4} \cup S_{q_6} \cup S_{q_7} \cup S_{q_9} = \{a, b\} \cup \{c, d, f, g\} \cup \{e, h, i\} \cup \{f, j, l\} \cup \{i, k, m\} \cup \{n\} = S$$

- (D) The reduction function f inputs an instance $\langle \mathcal{G}, B \rangle$ of the **Vertex Cover** problem. The computation is: let $S = \mathcal{E}$, the set of all edges. For each vertex v , construct $S_v \subseteq S$ consisting of the edges incident on v . Then the output of the function is an instance of the **Set Cover** problem, $f(\langle \mathcal{G}, B \rangle) = \langle S, S_{v_0}, \dots, S_{v_n} \rangle$. Clearly we can compute this function in polytime. And, as in the prior two items $\sigma = \langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ if and only if $f(\sigma) = \langle S, S_{v_0}, \dots, S_{v_n} \rangle \in \mathcal{L}_1$, since each vertex cover corresponds to an index set for a set cover.

V.6.25

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, v_0, v_1, B \rangle \mid \text{there is path from } v_0 \text{ to } v_1 \text{ of total weight at most } B \}$$

- (A) The **Unweighted Shortest Path** problem is an optimization problem, calling for the shortest path if one exists. As with many such problems, to express it as a language decision problem we use a parameter bound B_1 (the graph $\mathcal{G}$ is unweighted.)

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, v_0, v_1, B_1 \rangle \mid \text{there is path from } v_0 \text{ to } v_1 \text{ with at most } B_1 \text{ edges} \}$$

(In contrast, **Vertex-to-Vertex Path** is a decision problem: it just wants a ‘yes there is a path’ or ‘no there is not’ answer.)

The reduction function takes input $\langle \mathcal{G}_1, v_0, v_1, B_1 \rangle$ for the **Unweighted Shortest Path** problem, that is, $\mathcal{G}_1$ is unweighted. It constructs a weighted graph $\mathcal{H}_1$ out of $\mathcal{G}_1$ ’s vertices and edges, by giving each edge a weight of 1. The reduction function’s output is $\langle \mathcal{H}_1, v_0, v_1, B_1 \rangle$. If there is a path from $\mathcal{H}_1$ ’s vertex v_0 to its v_1 of total weight at most B_1 then there is an unweighted path with at most B_1 -many edges. Clearly this reduction function runs in polytime.

(B) The Longest Path problem is an optimization problem so this is the canonical language.

$$\mathcal{L}_2 = \{ \langle \mathcal{G}, v_0, v_1, B_2 \rangle \mid \text{there is path from } v_0 \text{ to } v_1 \text{ having total weight at least as large as } B_2 \}$$

To reduce it to the Shortest Path problem we produce a reduction function. It inputs $\langle \mathcal{G}, v_0, v_1, B_2 \rangle$, where $\mathcal{G}$ is a weighted graph. It converts $\mathcal{G}$ to a new graph $\mathcal{H}$ with the same vertices and edges but whose edge weights are the negation of $\mathcal{G}$'s edge weights. Then the output of the reduction function is $\langle \mathcal{H}, v_0, v_1, -B_2 \rangle$. This function clearly runs in polynomial time. Clearly also, a path that is short for the negatives of the weights is long for the original weights.

V.6.26

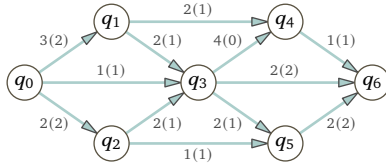
(A) Max-Flow has this language.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \text{there is a flow meeting the constraints of } \mathcal{G} \text{ such that the flow into the sink is at least } B \}$$

This is the language for the Linear Programming optimization problem.

$$\mathcal{L}_1 = \{ \langle C, Z, D \rangle \mid \text{there is a point } \vec{x} \text{ that meets the constraints } C \text{ and such that } Z(\vec{x}) \geq D \}$$

(B) There are algorithms to solve these problems but small ones we can do by eye. The total limit on the sink is five, and we can move from right to left across the graph. Here is the graph showing the capacity of each edge, and also showing its attained flow in parentheses.



(c) The variables are $x_{0,1}, \dots, x_{5,6}$. These are the capacity constraints.

$$\begin{aligned} 0 \leq x_{0,1} \leq 3, \quad 0 \leq x_{0,2} \leq 2, \quad 0 \leq x_{0,3} \leq 1, \quad 0 \leq x_{1,3} \leq 2, \quad 0 \leq x_{1,4} \leq 2, \\ 0 \leq x_{2,3} \leq 2, \quad 0 \leq x_{2,5} \leq 1, \quad 0 \leq x_{3,4} \leq 4, \quad 0 \leq x_{3,5} \leq 2, \quad 0 \leq x_{3,6} \leq 2, \\ 0 \leq x_{4,6} \leq 1, \quad \text{and} \quad 0 \leq x_{5,6} \leq 2 \end{aligned}$$

(These fit the definition of the problem because each is a combination of two linear constraints. For instance, $0 \leq x_{0,1} \leq 3$ is a combination of $0 \leq x_{0,1}$ and $x_{0,1} \leq 3$.) These are the flow constraints.

$$\begin{aligned} x_{0,1} &= x_{1,3} + x_{1,4}, & x_{0,2} &= x_{2,3} + x_{2,5}, & x_{0,3} + x_{1,3} + x_{2,3} &= x_{3,4} + x_{3,5} + x_{3,6}, \\ x_{1,4} + x_{3,4} &= x_{4,6}, & \text{and} & & x_{2,5} + x_{3,5} &= x_{5,6} \end{aligned}$$

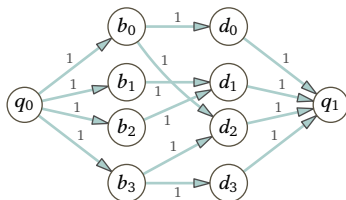
(These also fit the definition of the problem because each is a combination of two linear constraints. For instance, $x_{0,1} = x_{1,3} + x_{1,4}$ is a combination of $0 \leq x_{0,1} - x_{1,3} - x_{1,4}$ and $x_{0,1} - x_{1,3} - x_{1,4} \leq 0$.) We want to maximize $Z = x_{3,6} + x_{4,6} + x_{5,6}$.

(D) We must construct a reduction function f . The input is a pair $\langle \mathcal{G}, B \rangle$. Use that information to construct variables, capacity constraints, and flow constraints C , and an optimization expression Z . Then flows across the network correspond to feasible points $\vec{x}$ in the Linear Programming instance. So $\sigma = \langle \mathcal{G}, F, B \rangle \in \mathcal{L}_0$ if and only if $f(\sigma) = \langle C, Z, \vec{x}, B \rangle \in \mathcal{L}_1$. Clearly the reduction function can do the construction in polytime.

V.6.27

(A) The maximum number of matches is three, as in (b_0, d_0) , (b_1, d_1) , and (b_3, d_2) .

(B) Here is a network diagram. Each edge has weight 1.



(c) The **Max-Flow** problem has this language, where W is a bound.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, W \rangle \mid \text{a flow meets the constraints of } \mathcal{G} \text{ and the flow into the sink is at least } W \}$$

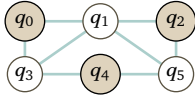
The **Matching** problem has this, where $B = \{b_0, \dots, b_{k-1}\}$.

$$\mathcal{L}_1 = \{ \langle B, S_0, \dots, S_{k-1}, X \rangle \mid \text{for } i \in I \text{ each } b_i \text{ has a match where } I \subseteq \{0, \dots, k-1\} \text{ has } |I| \geq X \}$$

(d) A reduction function f inputs a sequence $\sigma = \langle B, S_0, \dots, S_{k-1}, X \rangle$. As in the second item above, it constructs a network that from left to right has a source node, a bank of all the band nodes, a bank of all the drummer nodes, and a sink node. For each b_j and each member of S_j there is an edge, directed left to right and of weight 1. The output of the reduction function is the pair $f(\sigma) = \langle \mathcal{G}, X \rangle$. Then by construction, $\sigma \in \mathcal{L}_1$ if and only if $f(\sigma) \in \mathcal{L}_0$. Clearly we can compute this f in polytime.

V.6.28

(A) The natural independent set is this.



(B) This is the language for it as a language decision problem.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ has at least } B \text{ vertices that do not share an edge} \}$$

(c) This is the language for the **3-SAT** problem.

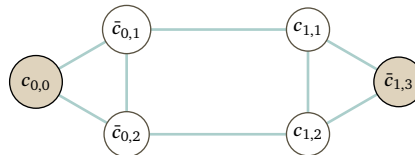
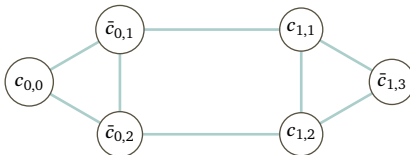
$$\mathcal{L}_1 = \{ E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals} \}$$

(d) Here is the truth table.

P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

Any of the lines ending in T shows that it is satisfiable.

(E) On the left is the graph.



On the right is the same graph with an independent set shown (there are other maximal independent sets, such as $\{\bar{c}_{0,1}, c_{1,2}\}$).

Taking $P_0 = T$ and $\neg P_3 = T$ (that is, $P_3 = F$) forces the expression E to evaluate to T . It doesn't matter what truth values we assign to P_1 and P_2 .

- (F) The reduction function f inputs an expression $\sigma = E$ that is an instance of **3-SAT**. The development in Example 6.5 points out that a CNF formula is satisfiable if and only if at least one literal in each clause evaluates to T . Literals from different clauses are compatible if they are not based on the same Boolean variable P_i , or if they are based on the same P_i and either both are ' P_i ' or both are ' $\neg P_i$ '.

So f builds the graph as described in the prior item. The vertices associated with clause i are: $c_{i,j}$ if clause i has a literal P_j , and $\bar{c}_{i,j}$ if clause i has $\neg P_j$. As to edges, for any clause C_i connect together all of the vertices $c_{i,j}$ from that clause to make a triangle (or degenerate triangle). In addition, where $i \neq \hat{i}$ connect all vertices of the forms $c_{i,j}$ and $\bar{c}_{\hat{i},j}$, since they arise from literals that are incompatible (P_j and $\neg P_j$).

Then $f(\sigma)$ is the problem instance that is the graph. Clearly the function can construct that instance in polytime.

As to showing that $\sigma \in \mathcal{L}_1$ if and only if $f(\sigma) \in \mathcal{L}_0$, recall that an independent set $I \subseteq \mathcal{N}$ is one where for each edge at most one vertex is in I . Because the vertices associated with each clause are all connected, the fact that a set is independent implies that at most one vertex in that grouping is in that set. The fact that conflicting vertices are connected means that at most one of them is selected. So the graph has an independent set containing as many vertices as there are clauses if and only if the underlying expression is satisfiable.

V V.6.29

- (A) This is a suitable language.

$$\mathcal{L} = \{ \langle I_0, \dots, I_{k-1} \rangle \mid \text{there is a } \vec{s} \text{ of integers that satisfies the inequalities } I_0, \dots \}$$

- (B) These constraints ensure in this integer programming context that each variable is either 0 or 1.

$$0 \leq x_0, \quad x_0 \leq 1, \quad 0 \leq x_1, \quad x_1 \leq 1, \quad 0 \leq x_2, \quad x_2 \leq 1,$$

A suitable linear inequality is $x_0 + (1 - x_1) + (1 - x_2) \geq 1$.

- (c) We must produce a reduction function f that inputs a propositional logic expression and that outputs a member of $\mathcal{L}$. For each variable P_i in that expression, create an integer variable x_i . as in the prior item, to force each x_i to be either 0 or 1, include the linear constraints $0 \leq x_i$ and $x_i \leq 1$. Also as in the prior item, for every clause in the propositional logic expression, add a constraint to the problem instance as in the prior item.

$$P_i \vee \neg P_j \vee \neg P_k \iff 1 \leq x_i + (1 - x_j) + (1 - x_k)$$

(It is the sum of terms where if the literal p_m is P_m then the term is x_m , while if the literal is $p_m = \neg P_m$ then the term is $(1 - x_m)$.) Clearly $\sigma \in \mathbf{3-SAT}$ if and only if $f(\sigma) \in \mathcal{L}$. Also clear is that we can write f to run in polytime.

V V.6.30

- (A) It is the decision problem for this language.

$$D = \{ p \mid p \text{ is a multivariable polynomial such that there is } \vec{c} = \langle c_0, \dots, c_{n-1} \rangle \in \mathbb{Z}^n \text{ so that } p(\vec{c}) = 0 \}$$

- (B) For reference, $S_{E_0} = \{ x_0(1 - x_0), x_1(1 - x_1), x_0(1 - x_1) \}$.

Let all three have a value of 0. The first polynomial $x_0(1 - x_0)$ evaluates to 0 if and only if x_0 is either 0 or 1. The second polynomial gives the same for x_1 . The third evaluates to 0 if and only if either $0 = x_0$ or $1 = x_1$.

The converse, that if all variables equal either 0 or 1 and either $0 = x_0$ or $1 = x_1$ then all three polynomials evaluate to 0, is clear.

- (c) For $E_1 = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$ the analogous set is this.

$$S_{E_1} = \{ x_0(1 - x_0), x_1(1 - x_1), x_2(1 - x_2), x_3(1 - x_3), x_0(1 - x_1)(1 - x_2), x_1x_2(1 - x_3) \}$$

The first four polynomials ensure that each variable takes on a value of either 0 or 1. The final two polynomials match the two clauses in E_1 .

- (D) The sum of squares has a value of zero if and only if each term has a value of zero.

$$p(x_0, x_1, x_2, x_3) = [x_0(1 - x_0)]^2 + [x_1(1 - x_1)]^2 + [x_2(1 - x_2)]^2 + [x_3(1 - x_3)]^2 \\ + [x_0(1 - x_1)(1 - x_2)]^2 + [x_1x_2(1 - x_3)]^2$$

- (E) We must produce the reduction function f that inputs an instance of 3-SAT, a Boolean expression E in which all clauses have three or fewer literals, and outputs a polynomial instance p of D. We will verify that E is satisfiable if and only of $p(\vec{s}) = 0$ for some integer n -tuple $\vec{s}$. And, f must run in polytime.

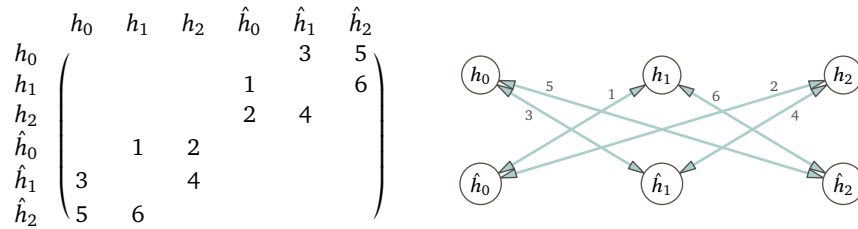
Obviously we follow the pattern in the prior items. For each variable P_i used in the input expression E , put $x_i(1 - x_i)$ into the set S_E . For each three-literal clause $L_{i_0} \vee L_{i_1} \vee L_{i_2}$ also add to S_E a polynomial: if $L_i = P_i$ then this polynomial has the factor x_i , while if the literal is a negation $L_i = \neg P_i$ then this polynomial has the factor $(1 - x_i)$. Finally, overall, the polynomial p output by the reduction function f is the sum of the squares of the polynomials from the set S_E .

For one direction of the verification, suppose that E is satisfiable. For all atoms P_i that are assigned T , take $x_i = 0$, while if P_i is assigned F then take $x_i = 1$. As a result, each polynomial member of S_E has the value of 0. Consequently the sum of squares polynomial p has a value of 0, and so the tuple of x_i 's is a zero of that polynomial.

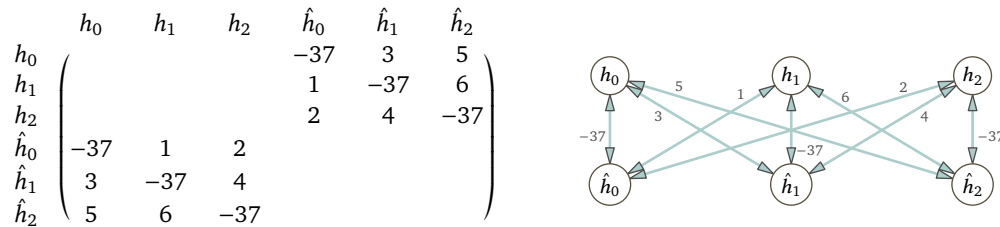
Now for the other direction. Suppose that we have a tuple of integers $\vec{c}$ such that $p(\vec{c}) = 0$. Because p is a sum of squares, each term must have a value of 0. The terms of the form $x_i(1 - x_i)$ imply that either $x_i = 0$ or $x_i = 1$. The terms that match the form of a clause imply that there is a corresponding assignment to the Boolean variables P_i that makes the clause true (namely, $x_i = 0$ is associated with $P_i = T$, and $x_i = 1$ is associated with $P_i = F$).

V.6.31

- (A) The reduction function inputs an instance of the symmetric problem and returns an instance of the asymmetric problem. A symmetric instance is a special case of an asymmetric instance. So the reduction function can be the identity function.
- (B) This is the graph matrix for the first stage of constructing $\mathcal{H}$. It is symmetric along the upper left to lower right diagonal. Also there is the matchinggraph of the first stage of $\mathcal{H}$. (It emphasizes the symmetry of the edges by showing arrows on both ends, although we usually show edges in such an undirected graph without any arrows at all.)



- (C) The adjusted matrix, the graph matrix for $\mathcal{H}$, is also symmetric.



- (D) The reduction function inputs instances σ of the Asymmetric Traveling Salesman problem, and outputs instances $f(\sigma)$ of the Symmetric Traveling Salesman problem.

We first make the argument for $\sigma = \mathcal{G}$ where $\mathcal{G}$ is graph pictured in the question body and $f(\sigma)$ is the symmetric graph $\mathcal{H}$ derived in the prior items. To make a circuit in $\mathcal{G}$, the only possibilities are going counterclockwise and going clockwise. Counterclockwise, $g_0 \rightarrow g_1 \rightarrow g_2 \rightarrow g_0$, costs 10. Clockwise costs 11. So the least-expensive circuit costs 10.

Observe that twice the number of vertices in $\mathcal{G}$ equals the number of vertices in $\mathcal{H}$, and so w is the number of edges in $\mathcal{H}$ multiplied by the maximum edge weight in $\mathcal{H}$. We claim that including in $\mathcal{H}$ edges with the large negative weights $-w - 1$ ensures that those edges must be part of any minimum circuit. To prove that, first observe that to visit every vertex, a circuit must have a number of edges equal to the number of vertices, which here is six edges (there is a correspondence between the edges and the vertex at the end of that edge). A circuit that does not use any edge with weight -37 has total weight of at least $6 \cdot 1$. Including one positive weight with -37 means that any circuit now has total weight between $-37 \cdot 1 + 5 \cdot 6 = -7$ and $-37 \cdot 1 + 5 \cdot 1 = -32$, so that lowers the total weight. From there, replacing another positive weight edge means that circuit goes to having a total weight between $(-37) \cdot 2 + 4 \cdot 6 = -50$ and $(-37) \cdot 2 + 4 \cdot 1 = -70$, an improvement. Finally, every circuit containing all three -37 edges totals while replacing a positive weight edge means that a circuit can weigh at most $(-37) \cdot 3 + 6 \cdot 3 = -93$. So three -37 's is unbeatable and by inspection there is such a circuit.

Therefore $\mathcal{H}$'s optimal solution is $h_0 \rightarrow \hat{h}_0 \rightarrow h_1 \rightarrow \hat{h}_1 \rightarrow h_2 \rightarrow \hat{h}_2 \rightarrow h_0$. It alternates between edges of the form $h_i \hat{h}_i$ and those of the form $\hat{h}_i h_j$, where edges with matching subscripts $g_i g_j$ are in $\mathcal{G}$'s optimal circuit.

This argument generalizes to any number of edges in $\mathcal{G}$. Let v be the number of vertices in $\mathcal{G}$ and let m be its maximum edge weight, so that $w = vm$. Replacing a positive weight edge will change what a circuit can weight by $-w - 1 = -vm - 1$, and the remaining edges in any circuit using this replaced edge have a total weight less than vm , so there is a net decrease. And there is a circuit using v -many edges of weight $-w - 1$.

The reduction function can clearly do the change from $\mathcal{G}$ to $\mathcal{H}$ in polytime.

V.6.32

(A) A little time playing with the table numbers

$C(t_i, w_j)$	w_0	w_1	w_2	w_3
t_0	13	4	7	6
t_1	1	11	5	4
t_2	6	7	2	8
t_3	1	3	5	9

shows that the optimum solution is to pair t_0, w_1 , along with t_1, w_3 , and t_2, w_2 , and finally t_3, w_0 . We can check that with a routine.

```
(define cost-table
  #[[13 4 7 6]
     #[1 11 5 4]
     #[6 7 2 8]
     #[1 3 5 9]])

(define (get-cost i j cost-table)
  (vector-ref (vector-ref cost-table i) j))

;; Find minimum cost of matching workers with tasks
;; For every permutation of task indices, make a list of costs,
;; then add, and then find the min.
(define (minimize-assignment cost-table)
  (apply min
    (for*/list ([task-perm (in-permutations '(0 1 2 3))])
      (apply + (map (lambda (i j) (get-cost i j cost-table))
                    '(0 1 2 3)
                    task-perm))))))
```

The total cost is 11.

```
> (minimize-assignment cost-table)
11
```


(B) The language for the Assignment problem is this.

$$\mathcal{L}_0 = \{ \langle W, T, C, B \rangle \mid W \text{ and } T \text{ are same-sized sets and } C: W \times T \rightarrow \mathbb{R}, \\ \text{and there is an assignment } a: W \rightarrow T \text{ costing no more than } B \}$$

This is the language for the Asymmetric Traveling Salesman problem.

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, D \rangle \mid \mathcal{G} \text{ is a directed weighted graph having a circuit having total weight no more than } D \}$$

A reduction function inputs Assignment problem instances $\langle W, T, C, B \rangle$ and outputs instances $\langle \mathcal{G}, D \rangle$ of the Asymmetric Traveling Salesman problem. As in the hint, take the output graph $\mathcal{G}$ to have two sets of vertices: one set labeled with each w_i and one labeled with each t_j . There are no inter- w edges or inter- t edges. For each pair w_i, t_j there is a directed edge in each direction. Give the edge from w_i to t_j a weight of $C(w_i, t_j)$. Give the edge from t_j to w_i a weight of 0.

If the input $\sigma = \langle W, T, C, B \rangle$ is an element of $\mathcal{L}_0$ then the total cost of some assignment a does not exceed B . Let that assignment be $w_0 \mapsto t_{i_0}, w_1 \mapsto t_{i_1}$, etc. Then in $\mathcal{G}$ the circuit starting at the vertex labeled w_0 and traveling to the vertex t_{i_0} , then to w_1 and t_{i_1} , etc. until returning back to w_0 , will be of total weight less than $D = B$. That is, $f(\sigma) = \langle \mathcal{G}, B \rangle$ is an element of $\mathcal{L}_1$.

Clearly we can write the reduction function associating this input and output to run in polytime.

V V.6.33

- (A) This has nothing to do with the ‘polytime’. If they were reducible then there would be a function f such that $n \in \mathbb{N}$ if and only if $f(n) \in \emptyset$. But the former is always true while the latter is always false.
- (B) Suppose that there is a computable function f such that $x \in A$ if and only if $f(x) \in B$, and suppose that B is computably enumerable. Then computably enumerate the set A by dovetailing an enumeration of B with an enumeration of the range of f : $f(0), f(1)$, etc. When an element appears on both lists, $f(i) = b_j$, then enumerate i into A .

As to $K^c \not\leq_p K$, the set K is computably enumerable but K^c is not.

- (C) Assume that $\mathcal{L} \leq_p \mathcal{L}^c$. Then there is a polytime f so that $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \mathcal{L}^c$. That’s equivalent to $\sigma \notin \mathcal{L}$ if and only if $f(\sigma) \notin \mathcal{L}^c$, which holds when $\sigma \in \mathcal{L}^c$ if and only if $f(\sigma) \in \mathcal{L}$.

V V.6.34

- (A) Reflexivity is easy; we must show that $\mathcal{L} \leq_p \mathcal{L}$ for any $\mathcal{L} \in \mathcal{P}(\Sigma^*)$. Take the reduction function to be the identity, $f(\sigma) = \sigma$. Clearly it can be computed in polytime, and just as clearly $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \mathcal{L}$. For transitivity let $\mathcal{L}_0 \leq_p \mathcal{L}_1$ via the function f and let $\mathcal{L}_1 \leq_p \mathcal{L}_2$ via the function g , which are polytime computable. Then for all $\sigma \in \Sigma^*$ we have $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$, and $f(\sigma) \in \mathcal{L}_1$ if and only if $g \circ f(\sigma) \in \mathcal{L}_2$. Because the two functions are polytime, so is their composition.
- (B) We first characterize the problems that the empty set reduces to, where $\emptyset \leq_p \mathcal{L}$. If $\emptyset \leq_p \mathcal{L}$ then there is a reduction function f where

$$\sigma \in \emptyset \quad \text{if and only if} \quad f(\sigma) \in \mathcal{L} \quad (*)$$

Never is $\sigma \in \emptyset$ true so $\mathcal{L} \neq \Sigma^*$ because in that case $f(\sigma) \in \mathcal{L}$ would always be true. If we let $\mathcal{L} \neq \Sigma^*$ and take α to be a string that is not a member of $\mathcal{L}$ then the constant function $f(\sigma) = \alpha$ suffices as the reduction.

We next characterize which problems reduce to the empty set, where $\mathcal{L} \leq_p \emptyset$.

$$\sigma \in \mathcal{L} \quad \text{if and only if} \quad f(\sigma) \in \emptyset \quad (**)$$

Never is $f(\sigma) \in \emptyset$ and so never is $\sigma \in \mathcal{L}$. Restated, the only problem that reduces to the empty set is the empty set itself.

The reasoning for Σ^* is similar. To characterize which problems satisfy $\Sigma^* \leq_p \mathcal{L}$, consider the double implication: $\sigma \in \Sigma^*$ if and only if $f(\sigma) \in \mathcal{L}$. The $\sigma \in \Sigma^*$ clause is always true and so $\mathcal{L}$ cannot be the empty set. But any other set $\mathcal{L} \neq \emptyset$ will work since if $\alpha \in \mathcal{L}$ then the constant function $f(\sigma) = \alpha$ will do as a reduction function.

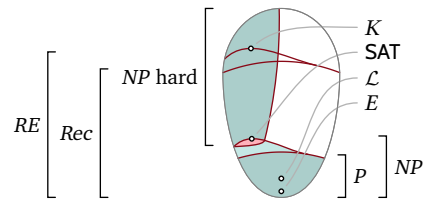
To characterize which sets satisfy $\mathcal{L} \leq_p \Sigma^*$ consider $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \Sigma^*$. The $f(\sigma) \in \Sigma^*$ clause is always true, so $\sigma \in \mathcal{L}$ must be always true, so the only such problem is $\mathcal{L} = \Sigma^*$.

- (c) Assume that $\mathcal{L}_0 \leq_p \mathcal{L}_1$, so that there is a polytime function $f: \Sigma^* \rightarrow \Sigma^*$ such that $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$. We will use this reduction function to show that $\mathcal{L}_1 \in NP$ implies that $\mathcal{L}_0 \in NP$.

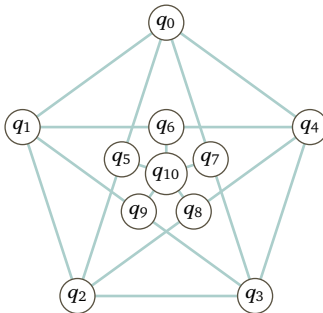
Assume that $\mathcal{L}_1 \in NP$. Then there is a nondeterministic Turing machine $\mathcal{P}_1$ that decides $\mathcal{L}_1$ (in particular, there is a polynomial p associated with this machine such that on input τ every maximal path terminates within $p(\tau)$ steps). For a machine $\mathcal{P}_0$ that decides $\mathcal{L}_0$, start with an input candidate string $\tau \in \Sigma^*$. Compute $f(\tau)$. Run $\mathcal{P}_1$ on input $f(\tau)$. If it accepts then $\mathcal{P}_0$ accepts, while if it rejects then $\mathcal{P}_0$ rejects. (And, observe that there is a polynomial q such that this nondeterministic algorithm has the property that every maximal path terminates in $q(|\tau|)$ steps).

- V V.6.35** We answer each with specific examples, but there are many other correct answers. For instance, for the first we can use any two members of P where one is a proper subset of the other.
- (A) Take $\mathcal{L}_0$ to be the language of strings whose length is a multiple of four, and $\mathcal{L}_1$ to be the language of strings whose length is even. Clearly $\mathcal{L}_0 \subset \mathcal{L}_1$, and both languages are in P so $\mathcal{L}_0 \leq_p \mathcal{L}_1$.
- (B) Take $\mathcal{L}_0$ to be the language of even-length strings, and $\mathcal{L}_1$ to be the language of odd-length strings. Clearly both languages are in P , so $\mathcal{L}_0 \leq_p \mathcal{L}_1$.
- (C) It is easy to see straight from the definition that the only set that polytime reduces to $\mathbb{B}^*$ is $\mathbb{B}^*$ itself. Take $\mathcal{L}_0$ to be the language of even-length strings and $\mathcal{L}_1 = \mathbb{B}^*$.
- (D) Similarly, the only set that polytime reduces to $\emptyset$ is $\emptyset$ itself. Take $\mathcal{L}_0$ to be the language of even-length strings and take $\mathcal{L}_1 = \emptyset$.

- V V.7.13** Here is the figure with the sets included.



- (A) Every computably enumerable set reduces to K . So the language K is at the top of RE .
- (B) Every set can compute the set E of even numbers in the time that it takes to read in the input (for that matter, just in the time it takes to read in the least significant bit of the input). So it has a $\mathcal{O}(n)$ algorithm and is very close to the bottom.
- (C) There are a variety of algorithms for the shortest path problem, but all are fast (quadratic or faster), so $\mathcal{L}$ is in the middle of P .
- (D) The **SAT** problem is NP complete so it is at the top of the NP problems.
- V V.7.14** Checking graphs for a 4-clique does indeed take time polynomial in the size of the graph. But there are ways for a graph to be not 3-colorable other than to have a 4-clique. For example, this graph has no triangles, no 3-cliques, and so there are no 4-cliques. But it is not 3-colorable.



- V V.7.15** The number one mistake is that “not computable in polynomial time” isn’t the criteria for membership in NP . Instead, the criteria for a problem being a member of NP is that it has a polytime verifier.

Another thing wrong is that the statement seems to confuse NP with the set difference $NP - P$. That is, your study partner seems to mean something like, “experts suspect that Satisfiability is a problem in NP but not in P .” (Along with that, they may well mean NP complete, not just an element of NP .)

One more thing wrong is that the statement seems to say that the problem is in that class because there is no algorithm yet known. It is not a question of whether we know of such an algorithm, it is a question of whether one exists.

- V **V.7.16** This particular algorithm is indeed not polynomial. But how to prove that there does not exist another algorithm, and that algorithm is polynomial? No one knows.
- V **V.7.17** In saying that the problem has a polytime algorithm, the commenter is saying that the problem is in P . But $P \subseteq NP$ and so this problem is in NP .
- V **V.7.18** Computational classes such as NP do not contain algorithms, they contain languages. (We don't distinguish too sharply between languages and problems, so may say that a problem is in a complexity class, but we do distinguish between a language and an algorithms to decide that language.)
- V **V.7.19**
 - (A) The statement “ NP is a subset of the NP complete sets” is false. The truth is the other way around: NP complete is a subset of NP . On the other hand, the contention that NP complete is a subset of NP hard is true. It is the intersection of NP hard and NP .
 - (B) Nondeterministic machines are a not a specialization of deterministic ones. Rather, nondeterministic machines are a generalization, that is, any deterministic machine is a nondeterministic one. (As to whether NP is a subset of P , that's trickier. As discussed in the section we don't know whether $P = NP$ is true, although most experts guess it is not, so we don't know whether $NP \subseteq P$.)
- V **V.7.20**
 - (A) True. Because $\text{Primality} \in NP$, it can be solved on a nondeterministic Turing machine. We can simulate such a machine with a deterministic one, and so we can at least in principle run that algorithm on currently-existing physical computers.
 - (B) False; we cannot infer this from the given facts. It may be that there is an efficient algorithm, but we don't know. (We don't know from the statement as given, and it is also not known to working researchers today.)
 - (C) False. The $\text{Traveling Salesman}$ problem is NP complete. Thus, from $\text{Prime Factorization} \in NP$, we can deduce that $\text{Prime Factorization} \leq_p \text{Traveling Salesman}$, and from an efficient algorithm for $\text{Traveling Salesman}$ we would get an efficient algorithm for $\text{Prime Factorization}$. But this is the reverse of what the question asks. The question says that $\text{Prime Factorization}$ is not known to be NP complete, so we don't know the other direction.
- V **V.7.21**
 - (A) False; for example, the brute force algorithm of checking all circuits solves all instances.
 - (B) True, where we use Cobham's Thesis to take 'quickly' to mean polytime and remembering this exercise's assumption that $P \neq NP$.
 - (C) False. If $\text{Traveling Salesman} \in P$ then every NP problem is in P , because every NP problem reduces to $\text{Traveling Salesman}$. That contradicts the $P \neq NP$ assumption.
 - (D) False. There is at least one algorithm, brute force, so this would imply that $\text{Traveling Salesman}$ is in P , which the prior item says is impossible.
- V **V.7.22**
 - (A) No, from $\mathcal{L}_1 \leq_p \mathcal{L}_0$ and $\mathcal{L}_0$ is NP complete, we cannot conclude that $\mathcal{L}_1$ is NP complete. An example is where $\mathcal{L}_0$ is the Satisfiability problem and $\mathcal{L}_1$ is the problem of determining whether an input string contains an even number of 1's.
 - (B) False; it may be that $\mathcal{L}_0$ is not a member of NP . An example is $\mathcal{L}_1 = \text{SAT}$ and the Halting problem set K .
 - (C) False. This is a reprise of the first item: from $\mathcal{L}_1 \leq_p \mathcal{L}_0$ and $\mathcal{L}_0$ is NP complete, we cannot conclude that $\mathcal{L}_1$ is NP complete, even knowing that $\mathcal{L}_1 \in NP$. An example is where $\mathcal{L}_0$ is the Satisfiability problem, and $\mathcal{L}_1$ is the problem of determining whether an input number is even.
 - (D) True; this is Lemma 7.4. If $\mathcal{L}_1$ is NP complete then any $\mathcal{L} \in NP$ is reducible to $\mathcal{L}_1$, so $\mathcal{L} \leq_p \mathcal{L}_1$ and transitivity of $\leq_p$ gives that $\mathcal{L} \leq_p \mathcal{L}_0$. That, along with the statement that $\mathcal{L}_0 \in NP$, shows that $\mathcal{L}_0$ is NP complete.
 - (E) False. For one thing, every problem reduces to itself, and if $\mathcal{L}_0 = \mathcal{L}_1$ then obviously if one is NP complete then both are. Less trivially, a problem is NP complete if it is in NP and every problem $\mathcal{L} \in NP$ reduces to it. So two NP complete problems reduce to each other. For example, any of the problems on the Basic List are reducible to any of the others, or to Satisfiability .

(F) False. An example is that $\mathcal{L}_1$ is the problem of determining whether a number is even, and $\mathcal{L}_0$ is a problem whose fastest solution algorithm is exponential.

(G) True. This is Lemma 6.9, that P is closed downward.

V **V.7.23** We just give one answer for each but other answers might be correct. (If $P = NP$ then all nontrivial sets are NP hard, but these answers apply also if $P \neq NP$.)

(A) SAT

(B) $E = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents in binary an even number}\}$

(C) SAT

(D) K (More precisely, the set of bitstring representations of numbers in K .)

(E) The complement of the set in the prior item.

(F) $\emptyset$

V **V.7.24** Assume that $P = NP$. Fix a language $\mathcal{L} \in P$ that is nontrivial, so $\mathcal{L} \neq \emptyset$ and $\mathcal{L} \neq \mathbb{N}$. Because $P \subseteq NP$ we have $\mathcal{L} \in NP$.

What remains is to show that $\mathcal{L}$ is NP hard, that every language $\hat{\mathcal{L}} \in NP$ satisfies that $\hat{\mathcal{L}} \leq_p \mathcal{L}$. But with $P = NP$ then $\hat{\mathcal{L}} \in P$, and Lemma 6.9 says that $\hat{\mathcal{L}} \leq_p \mathcal{L}$.

V **V.7.25**

(A) The language is the set of strings $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents an even number in binary}\}$. This set is in P because σ represents an even number if and only if the final bit is 0. Thus it is in NP .

If $\mathcal{L}$ were NP complete then every $\hat{\mathcal{L}} \in NP$ would satisfy that $\hat{\mathcal{L}} \leq_p \mathcal{L}$, and Lemma 6.9 would give that $P = NP$, which contradicts the exercise assumption.

(B) This is like the prior item, except that this problem is not as trivially a member of P . To see that it is, suppose that we are given a graph $\mathcal{G}$. We can check for a size four vertex cover (a set of vertices that includes at least one endpoint for every edge) by checking all 4-tuples of vertices. That takes a loop with $\binom{k}{4} \in \mathcal{O}(k^4)$ iterations, where k is the number of vertices in $\mathcal{G}$.

V **V.7.26** Move the head left, write a 1, and halt. This is a constant time algorithm.

V **V.7.27** The natural thing to try is to prove that 3-Satisfiability $\leq_p$ 4-Satisfiability.

The reduction function is the identity function. It inputs a clause with at most three literals, and outputs the same clause (which trivially has at most four literals). Clearly the input is satisfiable if and only if the output is satisfiable. Clearly also this reduction function runs in polytime.

V **V.7.28**

a	b	c	n_0	n_1	n_2	n_3	n_4	n_5
F	F	T	F	G	T	T	G	F
F	T	F	T	F	T	G	F	G
F	T	T	F	G	T	T	G	F
T	F	F	T	F	T	F	G	G
T	F	T	F	G	T	G	T	F
T	T	F	T	F	T	F	G	G
T	T	T	T	F	T	F	G	G

V **V.7.29**

(A) The sum of the elements of $\hat{S} = \{4, 6, 7, 13\}$ is $T = 30$.

(B) The sum of elements in $\hat{A} = \{6, 7, 19\}$ equals the sum of elements in $A - \hat{A} = \{3, 4, 12, 13\}$.

(C) Suppose that we can partition a set $A \subset \mathbb{N}$ into $\hat{A} \subset A$ and $A - \hat{A}$, with equal sums. Then the total of all of the elements of A is twice the sum of the elements of $\hat{A}$, and so is even.

(D) The Subset Sum problem is the language decision problem for this language.

$$\mathcal{L}_0 = \{\langle S, T \rangle \mid \text{a subset } \hat{S} \subseteq S \text{ has } \sum_{s \in \hat{S}} s = T\}$$

The Partition problem is the language decision problem for this language.

$$\mathcal{L}_1 = \{\langle A \rangle \mid \text{there is } \hat{A} \subset A \text{ with } \sum_{a \in \hat{A}} a = \sum_{a \in A - \hat{A}} a\}$$

- (E) We must produce a polytime computable function that takes in $\sigma = \langle A \rangle$ and outputs $f(\sigma) = \langle S, T \rangle$, such that $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$.

Given $\sigma = \langle A \rangle$, consider the sum of its elements $x = \sum_{a \in A} a$. Where

$$f(\sigma) = \begin{cases} \langle S, x/2 \rangle & \text{-- if } x \text{ is even} \\ \langle S, x + 1 \rangle & \text{-- otherwise} \end{cases}$$

we have that $\sigma \in \mathcal{L}_1$ if and only if $f(\sigma) \in \mathcal{L}_0$, as required, and clearly f is computable in polytime.

- (F) The Partition problem is one of the NP complete sets listed in Theorem 7.5. So Subset Sum is NP hard. But Subset Sum is in NP since as a witness ω we can use the subset. So it is NP complete.

V V.7.30 The Knapsack problem is the decision problem for this language.

$$\{ \langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle \mid \text{there is } I = \{i_0, \dots, i_k\} \subseteq \{0, \dots, n-1\} \text{ so } \sum_{i \in I} w_i \leq B \text{ and } \sum_{i \in I} v_i \geq T \}$$

In brief (as in Exercise 5.34), the Knapsack problem is in NP because we can take the witness to be a set of indices $\{i_0, \dots, i_k\} \subset \{0, \dots, n-1\}$ naming items that together meet the constraints.

Finish by recalling Example 6.4, that Subset Sum $\leq_p$ Knapsack. Since the prior exercise shows that Subset Sum is NP hard, this shows that Knapsack is also NP hard.

V V.7.31 We will show that Vertex Cover $\leq_p$ Set Cover.

Input to the reduction function f is an instance of Vertex Cover, $\sigma = \langle \mathcal{G}, B \rangle$. For the output, $f(\sigma) = \langle S, \{S_0, \dots, S_n\}, B \rangle$ where S is the set of $\mathcal{G}$'s edges, and where for each vertex in $\mathcal{G}$ there is a subset consisting of all of the edges containing that vertex.

Observe that there is a vertex cover for σ if and only if the associated subsets cover the set. Clearly this reduction function can be written to take only time polynomial in $|\mathcal{G}|$.

V V.7.32

- (A) Vertex q_2 is a sink, so it must end the longest path: $\langle q_3, q_6, q_4, q_7, q_8, q_5, q_0, q_1, q_2 \rangle$.
 (B) Take $B \in \mathbb{N}$ to be the parameter and consider this family of languages.

$$\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ has a simple path of length at least } B \}$$

- (C) The reduction function inputs an instance $\langle \mathcal{G}, v_0, v_1 \rangle$ of the Hamiltonian Path problem. It creates a graph $\hat{\mathcal{G}}$ by starting with $\mathcal{G}$ and appending two vertices along with two edges. The two vertices are w_0 and w_1 . One edge is directed and goes from w_0 to v_0 , while the other edge is also directed and goes from v_1 to w_1 . Then the reduction function outputs the pair $\langle \hat{\mathcal{G}}, |\mathcal{G}| + 1 \rangle$, where $|\mathcal{G}|$ is the number of vertices in $\mathcal{G}$. A path in $\hat{\mathcal{G}}$ containing $|\mathcal{G}| + 1$ edges must be a Hamiltonian from w_0 to w_1 . Such a path in $\hat{\mathcal{G}}$ exists if and only if in $\mathcal{G}$ a Hamiltonian path exists from v_0 to v_1 . Obviously that reduction function can be written to run in time polynomial in $|\mathcal{G}|$.
 (D) The exercise before this one shows that the Hamiltonian Path problem is NP complete and that, along with the other items of this exercise, gives that Longest Path is NP complete.

V V.7.33

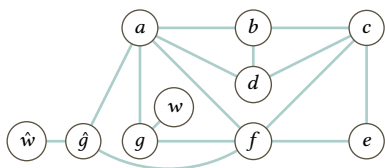
- (A) In short, the witness ω is the path.

At greater length, to apply Lemma 5.11 we will produce a deterministic Turing machine verifier, $\mathcal{V}$. It inputs pairs $\langle \mathcal{G}, \omega \rangle$, and must satisfy that if $\mathcal{G}$ has a Hamiltonian path then there is a witness ω such that $\mathcal{V}$ accepts the input pair, while if $\mathcal{G}$ does not have a Hamiltonian path then no witness will result in the pair being accepted. The verifier must run in polytime.

The verifier here interprets the witness ω to be a Hamiltonian path through the graph, so it checks that it is indeed a path in the given graph and that every vertex is in the path. If there is such a path then there is a witness that the routine can verify. If there is no such path then no string ω will suffice. The verifier clearly runs in polytime (the size of a graph is polynomial in the number of vertices since the number of edges is $\mathcal{O}(|\mathcal{V}|^2)$).

- (B) A Hamiltonian path is $\langle v_0, v_7, v_1, v_2, v_3, v_6, v_5, v_4, v_8 \rangle$. The two v_0 and v_8 must be the start and end of the path because they have degree 1.

- (c) In the starting graph a Hamiltonian circuit is $\langle g, a, d, b, c, e, f, g \rangle$. Here is the enhanced graph.



A Hamiltonian path is $\langle w, g, a, d, b, c, e, f, \hat{g}, \hat{w} \rangle$.

- (d) We must produce a reduction function f . It inputs a graph $\mathcal{G}$ and outputs a graph $f(\mathcal{G}) = \mathcal{H}$ with the property that $\mathcal{G}$ has a Hamiltonian circuit if and only if $\mathcal{H}$ has a Hamiltonian path.

As in the prior item, given $\mathcal{G}$, we form $\mathcal{H}$ by adding three vertices. First, fix a vertex $v \in \mathcal{G}$. Add a new vertex $\hat{v} \in \mathcal{H}$ that has the same connections as v , meaning that for every edge $v_i v_j$ add $\hat{v} v_i$ and for every edge of the form $v v_j$ add $\hat{v} v_j$. Also add two vertices of degree one: w connects only to v , and $\hat{w}$ connects only to $\hat{v}$.

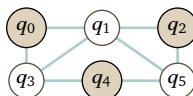
We must show that there is a Hamiltonian path in $\mathcal{H}$ if and only if there is a Hamiltonian circuit in $\mathcal{G}$. If there is a Hamiltonian circuit in $\mathcal{G}$ then the Hamiltonian path in $\mathcal{H}$ is derived by splitting the circuit at v , and hooking it to a path in $\mathcal{H}$ that starts at w , goes to v , then through the circuit, out to $\hat{v}$, and then to $\hat{w}$.

For the other direction suppose that there is a Hamiltonian path in $\mathcal{H}$. Then its start and end must be w and $\hat{w}$ because these vertices have degree 1. It must then proceed to v and $\hat{v}$. Because $\hat{v}$ is a copy of v , a Hamiltonian path in $\mathcal{H}$ that starts off at these vertices corresponds to a Hamiltonian circuit in $\mathcal{G}$.

- (e) The Hamiltonian Circuit problem is on the list of Basic NP Complete Problems, so the fact that it reduces to Hamiltonian Path means that Hamiltonian Path is NP hard. Because this exercise's first item shows that Hamiltonian Path is a member of NP, it is NP complete.

V.7.34

- (A) The natural independent set is this.



- (B) This is the language for the Independent Set problem.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ has at least } B\text{-many vertices that do not share an edge} \}$$

- (c) This is the language for 3-SAT.

$$\mathcal{L}_1 = \{ \langle E \rangle \mid E \text{ is a satisfiable CNF expression with at most three literals per clause} \}$$

- (D) See Figure 7 on page 47 for E 's truth table. One way to satisfy E is by taking $P_0 = T$ and $P_1 = P_2 = P_3 = F$.

- (E) The graph associated with the expression is this.



- (F) We must produce a reduction function, f . It inputs an instance of the 3-Satisfiability problem, E , and constructs a graph $\mathcal{G}$ as in the prior item. That is, for the j -th clause it adds three vertices, labeled with $v_{j,i}$ if the literal is P_i and with $\bar{v}_{j,i}$ if the literal is $\neg P_i$. It connects these three vertices completely, making a triangle. The reduction function also includes an edge for any two vertices labeled with the same second index i and where one has an overline and the other does not, and thus possibly making inter-triangular edges. Clearly this construction of $\mathcal{G}$ can happen in polytime.

We claim that an expression E with B -many clauses (and with at most three literals per clause) is satisfiable if and only if $\langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ where $\mathcal{G} = f(E)$.

First suppose that E is satisfiable. Then in every clause of E there is at least one literal that is T . Pick one such literal from each clause, denote the one from clause i with $\ell(i)$ and consider the set of associated vertices.

$$S = \{ v_{i,\ell(i)} \text{ or } \bar{v}_{i,\ell(i)} \mid \text{the associated literal in clause } i \text{ is } T \}$$

P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

FIGURE 7, FOR QUESTION V.7.34: Truth table for $E = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$.

We must verify that S has B distinct elements and is an independent set. It has B elements because each member is associated with a different clause of E , and so falls into a different triangle. It is an independent set because no two members are connected by an edge: in the construction there are two kinds of edges, those within a triangle and inter-triangular ones. No two members of S are in the same triangle so the first case is out. As for inter-triangular ones, they are between vertices associated with a literal and its negation. But a literal and its negation cannot both be T , so the associated vertices are not both members of S .

To finish, suppose that $\langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ and so $\mathcal{G}$ contains a size B independent set of vertices $\hat{S}$. We claim that taking the associated set of literals to be T will satisfy the expression E . Note that because the construction of $\mathcal{G}$ is such that there is an edge between vertices associated with a literal and its negation, we can set all of the associated literals to be T . Also note that because all vertices inside a triangle are connected, the B many members of $\hat{S}$ come from different triangles and so are associated with literals from the B many different clauses. So each clause has a literal that evaluates to T , and thus E is satisfied.

- V **V.7.35** No. If we knew of a problem in $NP - P$, complete or not, then we'd know that $P \neq NP$. We don't know that.

The section body mentions that assuming that that $P \neq NP$, Ladner's Theorem gives that there exist the problems asked for, but no naturally-occurring ones have been proved.

- V **V.7.36** For $\mathcal{L}_1$ take the set of bitstrings that start with $\emptyset$, $\{\emptyset \frown \sigma \mid \sigma \in \mathbb{B}^*\}$. Deciding membership is just a matter of deciding whether the first character is $\emptyset$, so $\mathcal{L}_1 \in P$.

For the other two, let $\mathcal{L}$ be any NP complete language of bitstrings. Take $\mathcal{L}_2 = \{\emptyset \frown \sigma \mid \sigma \in \mathcal{L}\}$ and $\mathcal{L}_0 = \mathcal{L}_1 \cup \{1 \frown \sigma \mid \sigma \in \mathcal{L}\}$.

- V **V.7.37** Assume that $\mathcal{L}_0$ is NP complete. Assume also that $\mathcal{L}_0 \leq_p \mathcal{L}_1$ and $\mathcal{L}_1 \in NP$. To verify that $\mathcal{L}_1$ is NP complete we must show that it is NP hard, that for every $\mathcal{L} \in NP$ we have $\mathcal{L} \leq_p \mathcal{L}_1$.

The assumption that $\mathcal{L}_0$ is NP complete gives $\mathcal{L} \leq_p \mathcal{L}_0$. Thus $\mathcal{L} \leq_p \mathcal{L}_0 \leq_p \mathcal{L}_1$ and transitivity of polytime reduction from Lemma 6.9 gives that $\mathcal{L} \leq_p \mathcal{L}_1$.

- V **V.7.38**

- (A) Fix $\mathcal{L}, \hat{\mathcal{L}} \in NP$. There are associated polytime verifiers $\mathcal{V}$ and $\hat{\mathcal{V}}$. A verifier for the union can just run those two on the input, and accept if either accepts.
- (B) Fix $\mathcal{L}, \hat{\mathcal{L}} \in NP$. There are associated polytime verifiers $\mathcal{V}$ and $\hat{\mathcal{V}}$. A verifier for the concatenation inputs a string σ , and loops over splitting it into substrings $\sigma = \sigma[0 : i] \frown \sigma[i :]$ for $i \in [0 .. |\sigma|]$, testing whether both verifiers accept. Each verifier is polytime, and running the loop keeps the overall algorithm runtime

inside of polytime.

- (c) The class P is closed under complement. Consequently, if NP is not closed under complement then $P \neq NP$. (However, our present state of knowledge is that it is also possible that NP is closed under complement while still being different than P .)

V **V.7.39** Countable. It is a subset of NP , which is countable because a verifier is a deterministic Turing machine, and there are countably many such machines.

V **V.A.10** They are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, and 97.

V **V.A.11**

- (A) We have $n = pq = 11 \cdot 13 = 143$ and $(p - 1) \cdot (q - 1) = 10 \cdot 12 = 120$
 (B) Many choices are possible. Here we will use $e = 7$.
 (c) Euclid's algorithm finds that the greatest common divisor is 1 in two lines.

$$120 = 7 \cdot 10 + 1$$

$$7 = 1 \cdot 7 + 0$$

Rewrite so that the greatest common divisor is a linear combination of the two.

$$1 = 120 + 7(-17)$$

Taking both sides modulo 120 gives $7 \cdot (-17) \equiv 1 \pmod{120}$. So, modulo 120, the multiplicative inverse of 7 is $120 - 17 = 103$.

- (D) The encryption is $m^e \pmod{n} = 9^7 \pmod{143} = 48$. The decryption is $48^{103} \pmod{143} = 9$.

V **V.A.12**

- (A) With two factors $n = k_0 \cdot k_1$ if one is greater than $\sqrt{n}$ then the other must be less.
 (B) If n is a perfect square $n = k \cdot k$ then its first nontrivial factor is $k = \sqrt{n}$.
 (C) The square root of 10^{12} is $10^6 = 1\,000\,000$, a million.
 (D) The input n has size about $\lg(n)$ bits. Thus $\sqrt{n} \approx (2^{\text{number digits of } n})^{1/2} \approx 2^{(1/2) \cdot \text{number digits of } n}$.

V **V.B.1** The main point relevant to this section is that for this problem we have very capable solvers, which will return the answer to even moderately large problems quite quickly on even a modest machine. The fact that a problem is NP hard does not mean that most of its not-too-large instances are slow to solve.

V **V.B.2** There are $n!$ circuits.

V **V.B.3** See Figure 8 on page 49. Basically, you can go around the outside for 14 or go through the middle for 15.

V.C.1 We can adjust the initial board to match the new one. Here are the first few lines.

```
; Exercise
(define INITIAL-CLAUSES
  (list (list '(1 2 3)) ; there is a 3 in position (1,2)
        (list '(1 3 4))
        (list '(1 4 5))
        (list '(1 6 6))
        (list '(1 7 9))
```

Then hitting the Run button in Dr Racket gives the output of all the clauses. This command drops them all to `sudoku.cfg`.

```
> (dump-to-file)
```

With the data in the external file we can run the command to implement the SAT solver.

```
jim@millstone:~/Documents/computing/src/scheme/complexity$ ./run-sat-solver.sh
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          729
| Number of clauses:           2467
| Parse time:                  0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:         0.00 s
|
```


<i>Vertex sequence</i>	<i>First edge</i>	<i>Second</i>	<i>Third</i>	<i>Return</i>	<i>Total</i>
$v_0-v_1-v_2-v_3$	3	1	4	7	15
$v_0-v_1-v_3-v_2$	3	5	4	2	14
$v_0-v_2-v_1-v_3$	2	1	5	7	15
$v_0-v_2-v_3-v_1$	2	4	5	3	14
$v_0-v_3-v_1-v_2$	7	5	1	2	15
$v_0-v_3-v_2-v_1$	7	4	1	3	15
$v_1-v_0-v_2-v_3$	3	2	4	5	14
$v_1-v_0-v_3-v_2$	3	7	4	1	15
$v_1-v_2-v_0-v_3$	1	2	7	5	15
$v_1-v_2-v_3-v_0$	1	4	7	3	15
$v_1-v_3-v_0-v_2$	5	7	2	1	15
$v_1-v_3-v_2-v_0$	5	4	2	3	14
$v_2-v_0-v_1-v_3$	2	3	5	4	14
$v_2-v_0-v_3-v_1$	2	7	5	1	15
$v_2-v_1-v_0-v_3$	1	3	7	4	15
$v_2-v_1-v_3-v_0$	1	5	7	2	15
$v_2-v_3-v_0-v_1$	4	7	3	1	15
$v_2-v_3-v_1-v_0$	4	5	3	2	14
$v_3-v_0-v_1-v_2$	7	3	1	4	15
$v_3-v_0-v_2-v_1$	7	2	1	5	15
$v_3-v_1-v_0-v_2$	5	3	2	4	14
$v_3-v_1-v_2-v_0$	5	1	2	7	15
$v_3-v_2-v_0-v_1$	4	2	3	5	14
$v_3-v_2-v_1-v_0$	4	1	3	7	15

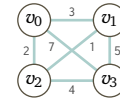


FIGURE 8, FOR QUESTION V.B.3: An exhaustive search for the Traveling Salesman problem.

===== [Search Statistics] =====								
Conflicts		ORIGINAL			LEARNT			Progress
		Vars	Clauses	Literals	Limit	Clauses	Lit/Cl	
	100	414	1928	6686	706	100	12	30.864 %
	250	414	1928	6686	777	250	12	30.864 %
	475	414	1928	6686	855	475	13	30.864 %
	812	414	1928	6686	940	812	13	30.864 %
	1318	414	1928	6686	1035	717	13	30.865 %
	2077	413	1921	6659	1138	1254	14	31.002 %
	3216	411	1914	6635	1252	980	13	31.277 %
	4924	400	1854	6458	1377	775	12	32.785 %
	7486	378	1780	6256	1515	1620	11	35.803 %
	11330	368	1664	5938	1666	1784	12	37.174 %
=====								
restarts	: 60							
conflicts	: 13476 (255867 /sec)							
decisions	: 21367 (0.00 % random) (405692 /sec)							
propagations	: 240090 (4558555 /sec)							
conflict literals	: 166762 (18.93 % deleted)							
Memory used	: 11.00 MB							
CPU time	: 0.052668 s							
SATISFIABLE								

Finally, in the file sat-solver.rkt are some more routines, including this one.

```
> (show-solved-board)
'#((7 3 4 5 1 6 9 2 8)
   #(8 1 5 4 9 2 7 6 3)
   #(6 9 2 3 8 7 4 1 5)
   #(9 4 6 8 2 3 1 5 7)
   #(1 2 8 7 5 4 3 9 6)
   #(5 7 3 1 6 9 2 8 4)
   #(2 8 7 6 3 1 5 4 9)
   #(3 5 1 9 4 8 6 7 2)
   #(4 6 9 2 7 5 8 3 1))
```

V.E.1 There are two halves. Here is the first. (We write n for $|\sigma|$.)

$$E_{1,0} = \bigwedge_{k \in [0..p(n)]} H_{-p(n),k} \vee \cdots \vee H_{p(n),k} = \bigwedge_{k \in [0..p(n)]} \bigvee_{i \in [-p(n)..p(n)]} H_{i,k}$$

The second involves pairs of predicates.

$$\begin{aligned} E_{1,1} &= \bigwedge_{k \in [0..p(n)]} (\neg S_{-p(n),k} \vee \neg S_{-p(n)+1,k}) \wedge \cdots \wedge (\neg S_{p(n)-1,k} \vee \neg S_{p(n),k}) \\ &= \bigwedge_{k \in [0..p(n)]} \bigwedge_{a < b \text{ for } a, b \in [0..|Q|-1]} \neg S_{a,k} \vee \neg S_{b,k} \end{aligned}$$

Then $E_1 = E_{1,0} \wedge E_{1,1}$.

V.E.2 We write $|\sigma|$ as n .

(A) Here also the overall expression has two halves.

This describes that at step $k = 0$ every cell from number $i = -5$ to 5 contains at least one of the three characters.

$$(C_{-5,0,0} \vee C_{-5,1,0} \vee C_{-5,2,0}) \wedge (C_{-4,0,0} \vee C_{-4,1,0} \vee C_{-4,2,0}) \wedge \cdots \wedge (C_{5,0,0} \vee C_{5,1,0} \vee C_{5,2,0})$$

For the full expression we must concatenate together one of those lines for every $k \in [0..p(n)]$.

A feel for the second half comes from this description that for step $k = 0$, cell number $i = -p(n)$ does not contain two symbols.

$$(\neg C_{-p(n),0,0} \vee \neg C_{-p(n),1,0}) \wedge (\neg C_{-p(n),0,0} \vee \neg C_{-p(n),2,0}) \wedge (\neg C_{-p(n),1,0} \vee \neg C_{-p(n),2,0})$$

The full expression needs three such clauses for each cell number $i \in [-p(n)..p(n)]$ and step $k \in [0..p(n)]$.

(B) We write it as two halves, so $E_2 = E_{2,0} \wedge E_{2,1}$. The first ensures that each cell contains at least one character.

$$E_{2,0} = \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} C_{i,0,k} \vee \cdots C_{i,|\Sigma|-1,k} = \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} \bigvee_{j \in [0..|\Sigma|-1]} C_{i,j,k}$$

The second ensures that no cell has more than one.

$$\begin{aligned} E_{2,1} &= \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} (\neg C_{i,0,k} \vee \neg C_{i,1,k}) \wedge \cdots (\neg C_{i,|\Sigma|-2,k} \vee \neg C_{i,|\Sigma|-1,k}) \\ &= \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} \bigwedge_{a < b \text{ for } a, b \in [0..|\Sigma|-1]} \neg C_{i,a,k} \vee \neg C_{i,b,k} \end{aligned}$$

V.E.3 The expression first contains this expression to enforce our convention that the machine's start state is q_0 .

$$S_{0,0} \wedge \neg S_{1,0} \wedge \cdots S_{|Q|-1,0}$$

Similarly it contains this expression

$$H_{-p(|\sigma|),0} \wedge \cdots \neg H_{-1,0} \wedge H_{0,0} \wedge \neg H_{1,0} \wedge \cdots H_{p(|\sigma|),0}$$

to enforce that at step $k = 0$ the head points to cell $i = 0$.

For the other clauses, let the input string be $\sigma = \langle s_{j_0}, \dots, s_{j_t} \rangle$ for $s_{j_0}, \dots, s_{j_t} \in \Sigma$. We have observed that with the transition function taking σ as input, only the cells numbered from $-p(|\sigma|)$ to $p(|\sigma|)$ are in play; all other cells have no role in the computation. (*Remark:* for this answer it is convenient to take the polynomial p to be such that $p(x) > x$ for all $x \in \mathbb{N}$. That way the initial string is sure to fit in the region from $-p(|\sigma|)$ to $p(|\sigma|)$.)

As in equation (*) of the section body discussion of 'conform', for cells with negative indices we include clauses that ensure the cell contains a blank and only a blank (we take blank to be symbol 0). Here is the clause for cell number $i = -p(|\sigma|)$.

$$C_{-p(|\sigma|),0,0} \wedge \neg C_{-p(|\sigma|),1,0} \wedge \cdots C_{-p(|\sigma|),|\Sigma|-1,0}$$

For cells with non-negative indices there are two cases. If the index is larger than the length of σ then we have a similar clause also ensuring that it contains blank and only blank. Here is the clause for cell number $p(n)$.

$$C_{p(|\sigma|),0,0} \wedge \neg C_{p(|\sigma|),1,0} \wedge \cdots C_{p(|\sigma|),|\Sigma|-1,0}$$

The other set of clauses cover cells that are populated with members of σ . For $i \in [0..|\sigma|-1]$ we have this.

$$\neg C_{i,0,0} \wedge \cdots \neg C_{i,j_i-1,0} \wedge C_{i,j_i,0} \wedge \neg C_{i,j_i+1,0} \cdots C_{i,|\Sigma|-1,0}$$

Appendices